Universidad Católica San Pablo

FACULTAD DE INGENIERÍA Y COMPUTACIÓN

DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

Escuela Profesional de Ciencia de la Computación

Arquitectura basada en representación intermedia para la generación confiable de consultas SQL/PGQ desde lenguaje natural

Tesis

Presentada por el bachiller

Guido Luis Tapia Oré

Para Optar por el Título profesional de:

Licenciado en Ciencia de la Computación

Asesor: Dra. Regina Paola Ticona Herrera

** Arequipa, Julio 2026 **

A mi familia por su incondicional apoyo en este proceso, a los profesores que fueron una guía en este proceso de investigación, a mis compañeros que ayudaron a no rendirme en este proceso.

Abreviaturas

API Application Programming Interface

LLM Large Language Model

PGQ Property Graph Queries

RBAC Role-Based Access Control

RDBMS Relational Database Management System

RI Representación Intermedia

RPQ Regular Path Query

SQL Structured Query Language

Agradecimientos

Aquí deberás colocar a quien y porque agradeces. Ejemplo:

En primer lugar deseo agradecer a Dios por haberme guiado a lo largo de estos cinco años de estudio.

Agradezco a mis padres por el apoyo brindado para forjarme como un profesional.

Agradezco a la universidad, mi *alma matter*, por haberme cobijado y brindado la formación que ahora me permitirá ayudar a construir una mejor sociedad.

Agradezco de forma muy especial a mi asesor Prof. Dr./Mag. nombre 1 por haberme guiado en esta tesis. ...

Deseo agradecer de forma especial a mis profesores: nombre 1, nombre 2, nombre 3 porque fueron ejemplos que deseo seguir en mi vida profesional.

Deseo agradecer al personal administrativo de la universidad: nombre 1, nombre 2, nombre 3. Muchas gracias por la atención brindada y porque siempre estuvieron dispuestas a ayudarnos.

Resumen

Aquí deberás colocar hasta 300 palabras como máximo, describiendo el problema que intentas resolver, la justificación, aportes o soluciones que planteas, qué tanto los haz alcanzado en calidad de resultados obtenidos.

Abstract

Here you should enter up to 300 words maximum, describing the problem you are trying to solve, the justification, contributions or solutions you are proposing, how much you have achieved them in terms of obtained outcomes.

Índice general

1. Introducción **2**

1.1. Motivación y Contexto 2

1.2. Planteamiento del Problema 4

1.3. Objetivos 4

1.3.1. Objetivos Específicos 5

1.4. Organización de la tesis 5

1.5. Cronograma 6

2. Fundamentos de Generación Text-to-Query y Consultas Híbridas **8**

2.1. Modelos de lenguaje y generación estructurada 8

2.1.1. Arquitectura transformer y decodificación autorregresiva 9

2.1.2. Prompting y aprendizaje en contexto 10

2.2. Modelos de datos y lenguajes de consulta 10

2.2.1. Modelo relacional y álgebra relacional 10

2.2.2. SQL como lenguaje declarativo 11

2.2.3. Grafos de propiedad y pattern matching 11

2.2.4. El estándar SQL/PGQ como integración 11

2.3. Semantic parsing y generación *text-to-query* 12

2.3.1. *Semantic parsing*: definición y marco general 12

2.3.2. *Text-to-SQL* como caso particular y extensión a *text-to-query* 12

2.4. Representaciones intermedias 13

2.4.1. Noción formal de representación intermedia 13
2.4.2. Propiedades deseables 13
2.5. Verificación de consultas generadas 14
2.5.1. Verificación estática 14
2.5.2. Verificación dinámica 15
2.5.3. Testing metamórfico 15

3. Arquitecturas y Enfoques Actuales en Text-to-Query 16
3.1. Marco taxonómico del análisis 16
3.2. Paradigma de aprendizaje 18
3.2.1. Aprendizaje con parámetros congelados 18
3.2.2. Aprendizaje con parámetros ajustados 20
3.2.3. Estrategias híbridas 21
3.2.4. Observaciones críticas 21
3.3. Arquitectura del Sistema 21
3.3.1. Arquitecturas monolíticas 21
3.3.2. Pipelines modulares 22
3.3.3. Arquitecturas multiagente 22
3.3.4. Arquitecturas agentic-search 23
3.3.5. Arquitecturas de generación con reordenamiento 23
3.3.6. Observaciones críticas 23
3.4. Estrategias de Representación 23
3.4.1. Codificación de la entrada 24
3.4.2. Representación intermedia 24
3.4.3. Estrategias de decodificación 26
3.4.4. Observaciones críticas 26
3.5. Mecanismos de Enriquecimiento Contextual 27
3.5.1. Schema linking 27

3.5.2. Recuperación de contenido de la base de datos 27
3.5.3. Conocimiento externo y de dominio 27
3.5.4. Auto-refinamiento 28
3.5.5. Observaciones críticas 28
3.6. Paradigma de Razonamiento 28
3.6.1. Razonamiento directo 28
3.6.2. Cadena de pensamiento 29
3.6.3. Razonamiento descompositivo 29
3.6.4. Razonamiento de múltiples caminos 29
3.6.5. Razonamiento de retroalimentación iterativa 29
3.6.6. Observaciones críticas 30
3.7. Propiedades Evaluativas Objetivo 30
3.7.1. Corrección 30
3.7.2. Eficiencia 30
3.7.3. Robustez 31
3.7.4. Adaptabilidad 31
3.7.5. Escalabilidad 31
3.7.6. Interpretabilidad 31
3.7.7. Observaciones críticas 32
3.8. Síntesis y Brecha de Investigación 32

4. Propuesta: Arquitectura basada en Representación Intermedia para Generación Confiable de Consultas SQL/PGQ 34

4.1. Bases técnicas y criterios de selección 34
4.2. Requisitos y principios de diseño 36
4.3. Flujo metodológico general 39
4.4. Arquitectura y componentes 40
4.4.1. Anclaje al esquema dual 41

4.4.2. Representación intermedia tipada (IR-SQL/PGQ) 41
4.4.3. Compilador determinista a SQL/PGQ 45
4.4.4. Bucle de verificación 45
4.5. Implementación y replicabilidad 47
4.5.1. Stack técnico y versiones declaradas 48
4.6. Resultados esperados 48
4.6.1. Resultados esperados al cierre del semestre 2026-1 48
4.6.2. Resultados esperados al cierre de la tesis 49
4.6.3. Criterios de éxito y limitaciones reconocidas 51
4.7. Síntesis y transición 52

5. Experimentos **53**

6. Resultados **54**

7. Conclusiones y Trabajos Futuros **55**
7.1. Problemas encontrados 55
7.2. Recomendaciones 55
7.3. Trabajos futuros 55

Bibliografía **61**

Índice de tablas

1.1. Cronograma de actividades de la tesis. 6

Índice de figuras

3.1. Taxonomía multidimensional propuesta. Cada uno de los seis ejes ortogonales agrupa las decisiones de diseño observadas en la literatura de *text-to-query* y sus principales subcategorías. 17

4.1. Bases técnicas reutilizadas para construir la propuesta. Cada base satisface los criterios comunes de selección y aporta a una dimensión específica del marco taxonómico definido en la Sección 3.1. 37

4.2. Flujo metodológico general del *pipeline* propuesto. Las cinco etapas secuenciales procesan la pregunta en lenguaje natural y el esquema dual (relacional y grafo de propiedad) hasta producir una consulta Structured Query Language (SQL)/Property Graph Queries (PGQ) verificada. 39

Capítulo 1

Introducción

Este capítulo establece el contexto, la motivación y el problema de investigación que aborda la presente tesis, cuyo enfoque central radica en el diseño de una arquitectura de razonamiento estructural, basada en Representación Intermedia (RI), para mitigar las alucinaciones en la generación *text-to-query* de consultas empresariales complejas, específicamente orientada al estándar híbrido SQL/PGQ.

Esta investigación se sitúa en la intersección entre los Large Language Models (LLMs), los sistemas de gestión de bases de datos relacionales (Relational Database Management Systems (RDBMSs)) y los lenguajes de consulta híbridos.

Como se evidenciará en el posterior desarrollo, el diseño propuesto se presenta como una respuesta estructurada al creciente interés de la industria por desplegar interfaces en lenguaje natural altamente fiables en entornos empresariales a gran escala.

1.1. Motivación y Contexto

En el contexto actual de la adopción de LLMs en procesos empresariales y el creciente interés por integrarlos en la optimización de flujos de trabajo, se evidencia una clara tendencia hacia interfaces basadas en lenguaje natural [Adedeji and Mabere, 2023], siendo preferidas por sobre interfaces gráficas tradicionales. Este cambio en la tendencia es particularmente notable en organizaciones *data-driven*, cuya toma de decisiones depende de la capacidad de responder preguntas de negocio en ciclos cortos y con altas garantías de calidad [Pérez et al., 2025]. En este marco, las interfaces en lenguaje natural para la consulta de datos han resurgido con fuerza, habilitando experiencias de *self-service analytics* las cuales están tomando mayor importancia en el mercado actual, siendo que estas permiten que perfiles no técnicos formulen preguntas en texto libre y lenguaje cotidiano, obteniendo resultados estructurados sin la necesidad de escribir manualmente consultas complejas [Quamar et al., 2022].

No obstante, esta tarea no es trivial. Y opuesto a lo que se podría intuir al los usuarios ver interfaces más sencillas, resulta mucho más compleja a nivel técnico y crítica al volver más difusas las capas de acceso a datos que suelen poseer los sistemas convencionales.

El despliegue de estos modelos en entornos de producción demanda el cumplimiento de múltiples condiciones, entre los cuales podemos incluir tres pilares críticos: (i) *integridad técnica*, que garantiza tanto la ejecutabilidad en el RDBMS como la fidelidad semántica de la consulta [Lei et al., 2025]; (ii) *seguridad y gobernanza*, mediante mecanismos de control de acceso basado en roles (Role-Based Access Control (RBAC)) y auditoría para prevenir la exfiltración de datos sensibles [Liu et al., 2025b]; y (iii) *robustez y escalabilidad del esquema*, asegurando que el modelo mantenga su precisión frente a catálogos de datos masivos, dinámicos y con nomenclaturas complejas [Tang et al., 2026]. La presente investigación se concentra en el primero de estos pilares.

Asimismo, también es importante tomar en cuenta las propias limitaciones de los modelos presentes en el mercado, pues a pesar del gran interés generado en los usuarios y de los avances acelerados que se han dado en los últimos años, los LLMs no son modelos perfectos y aún presentan fallas sistemáticas de *fidelidad y grounding* al generar artefactos formales. Al aplicarlos específicamente en la tarea de *text-to-query*, estas fallas se tienden a manifestar como referencias a tablas o columnas inexistentes, rutas de *join* inventadas, condiciones lógicas mal interpretadas y/o agregaciones incongruentes con el esquema. Lo cual si bien esta es una limitación general de los modelos de lenguaje, el riesgo se agrava severamente cuando el código se genera para un RDBMS: un error no solo afecta la exactitud analítica, sino que puede inducir sobrecostos computacionales, vulnerabilidades de seguridad e incluso modificar datos existentes.

Otro aspecto a considerar en la implementación de estos modelos es que investigaciones recientes demuestran que el alto desempeño en *benchmarks* clásicos decae drásticamente al enfrentarse a bases de datos corporativas reales, dado que los casos ideales que se tienden a ver son reemplazados por estructuras ruidosas y con flujos de trabajo de múltiples pasos [Li et al., 2023, Lei et al., 2025], lo cual refuerza la necesidad de transitar desde enfoques ingenuos entre los que encontramos estrategias de *prompting* monolíticos y arquitecturas puramente agénticas hacia sistemas más robustos y orquestados [Fürst et al., 2024].

En paralelo a lo ya mencionado, es importante considerar que el ecosistema de bases de datos está evolucionando, con una clara tendencia a una mayor interrelación y una preferencia hacia modelos híbridos. En la mayoría de dominios observados esto se refleja en el modelado de relaciones mediante grafos, dado que estos resulta más natural para expresar caminos y vecindarios complejos. No obstante, siendo las bases de datos relacionales las que dominan el entorno corporativo por características como su alta integridad de datos, estructura organizada en tablas y la madurez, es más bien la integración de *property graphs* lo que se presenta como una evolución natural de la industria. Lo ya mencionado se valida con la reciente estandarización ISO/IEC 9075-16:2023 (SQL/PGQ), la cual formaliza la consulta de grafos de propiedades directamente desde SQL [ISO/IEC JTC 1/SC 32, 2023]. Esta evolución, si bien unifica ambos paradigmas, también incrementa significativamente la dificultad de la tarea *text-to-query*, ya que el espacio de búsqueda se expande y la validación lógica se vuelve más rigurosa, multiplicando el riesgo de alucinaciones y presumiblemente comprometiendo el rendimiento de los modelos existentes.

Abordando lo previamente mencionado sobre la abundancia de propuestas poco sofisticadas, en la industria actual, las soluciones comerciales suelen apoyarse en estrategias heurísticas como inyectar el esquema en el *prompt* o aplicar ciclos de autocorrección mediante retroalimentación en texto libre, métodos que han demostrado ser frágiles ante

la ambigüedad semántica del usuario y el *schema drift* [Ding et al., 2026], entre más problemas.

Son todos estos antecedentes convergen en una observación central: si bien existen técnicas parciales para mejorar el rendimiento en *text-to-query* y múltiples propuestas dentro de esta área, aún existe un gran campo para investigar su implementación de forma sistémica y mejora, especialmente orientándonos a la integración con el estándar híbrido SQL/PGQ, el cual presenta mayores desafíos y que no ha sido suficientemente investigado al momento de realizada esta investigación. Siendo esta brecha la que constituye la principal motivación de la presente investigación.

1.2. Planteamiento del Problema

El problema presentado es *la generación de consultas a bases de datos a partir de lenguaje natural*, el cual se presenta como un problema de traducción entre dos espacios formalmente distintos: el espacio continuo y ambiguo del texto libre y el espacio discreto de un lenguaje de consulta con semántica determinista, restringido por un esquema de base de datos específico. En entornos empresariales, donde las decisiones operativas dependen de la corrección de estas consultas, el problema adquiere relevancia práctica directa, dado que una consulta incorrecta no solo produce resultados erróneos, sino que puede traducirse en sobrecostos y comprometer la integridad de los datos y operaciones realizadas.

El problema se agrava al considerar el estándar híbrido SQL/PGQ, donde la generación debe operar simultáneamente sobre dos estructuras formales superpuestas: el esquema relacional, el cual incluye tablas, columnas, restricciones, y el grafo de propiedad que representaría los nodos, aristas, propiedades clave-valor. En este contexto, los errores no solo afectan a cada estructura por separado, sino que surgen nuevas categorías de fallo derivadas de la incoherencia cruzada entre ambas. Esta dualidad contribuye a incrementar significativamente el espacio de decisiones del modelo y, con ello, la dificultad de garantizar la corrección de las consultas generadas.

Y si bien la literatura reciente ha propuesto técnicas parciales para abordar distintas facetas de este problema, los sistemas existentes han sido diseñados y evaluados sobre el régimen relacional puro; su comportamiento sobre el régimen híbrido SQL/PGQ está poco desarrollado y, en ausencia de mecanismos específicos para razonar sobre el componente de grafo, es correcto presumir que su desempeño no se mantenga al transferirse a este contexto. Esta brecha motiva la pregunta central de la presente investigación: *¿cómo mitigar de forma sistemática las alucinaciones estructurales y semánticas en la generación* text-to-query *sobre esquemas híbridos SQL/PGQ, de modo que las consultas resultantes sean correctas, verificables y operables en contextos empresariales?*

1.3. Objetivos

Objetivo general: Proponer una arquitectura de razonamiento estructural basada en RI para la generación *text-to-query* orientada al estándar SQL/PGQ, que mitigue las

alucinaciones de esquema y los errores semánticos en escenarios de consulta empresarial.

1.3.1. Objetivos Específicos

- Diseñar una RI unificada para consultas híbridas SQL/PGQ que integre componentes relacionales y de patrón de grafo con reglas de tipado y compilación determinista.
- Implementar un *pipeline* de razonamiento estructural que incorpore anclaje al esquema, generación con restricciones formales sobre el espacio de salida y un bucle de verificación estática y dinámica.
- Evaluar experimentalmente la arquitectura propuesta frente a métodos del estado del arte mediante métricas de corrección, tasa de alucinación, robustez y eficiencia sobre *benchmarks* establecidos.
- Caracterizar la transferibilidad de la arquitectura propuesta a entornos equivalentes y estimar el coste por consulta, en consistencia con los requisitos de portabilidad y coste acotado declarados en el Capítulo 4.

1.4. Organización de la tesis

El resto de este documento está organizado de la siguiente manera:

- El capítulo 2 presenta los fundamentos conceptuales y técnicos: LLMs y alucinaciones, *text-to-SQL*, el estándar SQL/PGQ, y las técnicas de RI, decodificación restringida y verificación.
- El capítulo 3 revisa las arquitecturas y enfoques actuales en *text-to-query* (pipelines por etapas, agentes, *constrained decoding*, detección y mitigación de alucinaciones) y posiciona el problema en el contexto empresarial y del estándar híbrido.
- El capítulo 4 describe la propuesta: la RI tipada para SQL/PGQ, el compilador de RI a SQL/PGQ, el pipeline de razonamiento estructural y el bucle de verificación (estático y por ejecución).
- El capítulo 5 detalla el diseño experimental: datasets/benchmarks, construcción de corpus híbrido para SQL/PGQ, métricas de alucinación y corrección, baselines y *ablations*.
- El capítulo 6 presenta y analiza los resultados obtenidos, incluyendo robustez a *schema drift*, seguridad/eficiencia, y evaluación de componentes, validando los resultados con datasets relevantes.
- El capítulo 7 concluye la tesis, resume aportes, limitaciones y líneas de trabajo futuro.

1.5. Cronograma

Tabla 1.1: Cronograma de actividades de la tesis.

		N.°	Actividad	Detalles	Semana	Fecha	%
Semestre 2026-1	Parcial	1	Alineamiento inicial del trabajo	Revisión del curso, entregables, estructura de la tesis y ajuste del tema con enfoque en SQL/PGQ + representación intermedia.	Sem. 1	11/03 – 14/03	2
		2	Definición final de tema y asesor	Alcance, contribución central y validación con asesora. Primera revisión de tabla survey.	Sem. 2	14/03 – 21/03	5
		3	Formalización y delimitación	Compromiso de asesoría, planteamiento del problema, objetivo general y específicos.	Sem. 3	21/03 – 28/03	10
		4	Consolidación del survey	Depuración del estado del arte, clasificación de trabajos, taxonomía y bibliografía base.	Sem. 4	28/03 – 01/04	15
		5	Redacción de Introducción v1	Motivación, contexto, problema, objetivos, organización y cronograma.	Sem. 5	01/04 – 08/04	18
		6	Cierre de Introducción	Ajustes con feedback y versión entregable del capítulo Introducción.	Sem. 5	08/04 – 11/04	20
		7	Redacción de Marco Teórico	Fundamentos teóricos, conceptos clave y modelos explicativos de la investigación.	Sem. 6	11/04 – 14/04	28
		8	Redacción de Estado del Arte	Literatura reciente, trabajos previos, vacíos de investigación y posicionamiento.	Sem. 6	14/04 – 18/04	35
		9	Capítulo Propuesta v1	Arquitectura, pipeline, representación intermedia, componentes y flujo de verificación.	Sem. 7	18/04 – 25/04	45
		10	Capítulo Propuesta v2	Protocolo experimental, datasets, métricas, baselines y criterios de evaluación.	Sem. 8	26/04 – 02/05	50
		11	Integración del Plan de Tesis	Consolidación de capítulos, coherencia interna, flujo narrativo y bibliografía final.	Sem. 9	03/05 – 09/05	55
		12	Plan de Tesis listo para envío	Cierre de documento, diapositivas y ensayo previo a la exposición.	Sem. 10	11/05 – 16/05	55
	Final	13	Implementación base experimental	Repositorio, estructura del prototipo, casos de prueba y baseline(s).	Sem. 11	17/05 – 23/05	62
		14	Implementación arquitectura v1	Flujo NL → RI → consulta estructurada/verificable.	Sem. 12	24/05 – 30/05	68
		15	Integración end-to-end y validación	Integración de componentes, validaciones estructurales y depuración.	Sem. 13	31/05 – 06/06	73
		16	Experimentación preliminar	Pruebas comparativas, recopilación de resultados y análisis preliminar.	Sem. 14	07/06 – 13/06	77
		17	Resultados al 80 y Avances	Prototipo funcional, resultados parciales sólidos y figuras iniciales.	Sem. 15	14/06 – 20/06	80
		18	Cierre del documento de Avances	Documento completo, conclusiones preliminares y preparación de diapositivas.	Sem. 16	21/06 – 27/06	80
		19	Envío y exposición de Avances	Presentación final del semestre con resultados parciales consolidados.	Sem. 17	28/06 – 04/07	80

Continúa en la siguiente página...

		N.°	Actividad	Detalles	Semana	Fecha	%
Int.		20	Revisión y fortalecimiento	Análisis de trabajos complementarios y refinamiento de la arquitectura propuesta.		10/07 – 10/08	82
Semestre 2026-2	Parcial	21	Proyecto de Investigación	Propuesta formal: impacto, viabilidad y posible aplicación institucional.	Sem. 1–2	12/08 – 22/08	84
		22	Ampliación experimental	Nuevos escenarios, ablations, análisis de errores y validación empresarial.	Sem. 3–5	23/08 – 15/09	87
		23	Resultados finales y discusión	Comparación con baselines, análisis crítico y discusión de aportes.	Sem. 6–8	16/09 – 06/10	90
		24	Preparación y Exposición Parcial	Presentación con metodología, resultados preliminares y validación ante jurado.	Sem. 9	07/10 – 13/10	91
	Final	25	Revisión y ajustes post-parcial	Correcciones metodológicas y refinamiento con feedback del jurado.	Sem. 10	14/10 – 20/10	92
		26	Síntesis de resultados y divulgación	Hallazgos en formato de reporte técnico o borrador de artículo.	Sem. 11	21/10 – 27/10	93
		27	Redacción final de tesis	Cierre de capítulos, conclusiones, trabajo futuro y referencias.	Sem. 12–14	28/10 – 15/11	96
		28	Correcciones con asesora	Ajustes de fondo y forma, normalización y aprobación final.	Sem. 15–16	16/11 – 30/11	98
		29	Preparación y Exposición Final	Diapositivas, ensayo de tiempos y sustentación final ante jurado.	Sem. 17–18	01/12 – 12/12	100

Capítulo 2

Fundamentos de Generación Text-to-Query y Consultas Híbridas

Este capítulo presenta los conceptos necesarios para comprender la propuesta del presente trabajo, la cual será desarrollada en el Capítulo 4. La exposición avanza de manera progresiva, desde los fundamentos de los modelos de lenguaje hasta los mecanismos que permiten controlar y verificar las consultas generadas.

En primer lugar, se describen los LLMs, su forma de generar texto y los tipos de desafíos que pueden aparecer en este proceso (Sección 2.1). Luego, se revisan los modelos de datos relacional y de grafos de propiedad, así como su integración mediante el estándar SQL/PGQ (Sección 2.2) y se introduce el problema de *semantic parsing* y, dentro de este, la tarea de *text-to-query* (Sección 2.3).

Posteriormente, se explica el papel de una representación intermedia tipada como puente entre la interpretación realizada por el modelo y la generación final de una consulta ejecutable (Sección 2.4). Finalmente, se revisan las formas de verificación estática y dinámica que permiten evaluar la corrección de la consulta generada (Sección 2.5).

Estos fundamentos sirven de base para el análisis del estado del arte y para la propuesta de esta investigación.

2.1. Modelos de lenguaje y generación estructurada

Los LLMs, o modelos extensos de lenguaje, son modelos basados en arquitecturas *transformer* [Vaswani et al., 2017] y entrenados sobre grandes volúmenes de texto, los cuales en los últimos años se han convertido en una base importante para la investigación en el área de *text-to-query* debido a su capacidad para interpretar preguntas en lenguaje natural y poder generar respuestas en formatos estructurados. Esto en adición a su flexibilidad para adaptarse a nuevos esquemas de bases de datos mediante aprendizaje en contexto [Brown et al., 2020] les permite generar consultas sin requerir un reentrenamiento completo del modelo para cada dominio o base de datos.

Esta capacidad de generalización se complementa con la versatilidad de los modelos preentrenados, los cuales pueden especializarse en la generación de consultas estructuradas mediante *fine-tuning*, diseño avanzado de *prompts* o enfoques híbridos que combinan ambas estrategias.

No obstante, esta capacidad no resuelve por sí sola el problema presentado, ya que estos modelos se caracterizan por generar texto de forma autorregresiva, es decir que se produce una secuencia de *tokens* prediciendo cada elemento a partir de los anteriores, siendo este un proceso estadístico, que no incorpora de manera nativa las reglas gramaticales ni el esquema sobre el cual operar, lo que abre la puerta a la posibilidad de producir consultas que parezcan correctas desde el punto de vista del lenguaje natural, pero que contienen errores estructurales, referencias inexistentes o interpretaciones equivocadas de esquema. Fenómeno que en la literatura se conoce como *alucinaciones* [Ji et al., 2023].

2.1.1. Arquitectura transformer y decodificación autorregresiva

La arquitectura *transformer* [Vaswani et al., 2017] permite procesar una secuencia de entrada $x = (x_1, \ldots, x_n)$ y representar cada *token* mediante vectores contextualizados. Para ello utiliza mecanismos de atención, los cuales permiten que el modelo relacione distintos elementos de la secuencia, incluso cuando se encuentran alejados entre sí. A diferencia de arquitecturas recurrentes, este procesamiento no depende de recorrer la secuencia paso a paso, sino de comparar los *tokens* entre sí mediante capas de atención.

En tareas de *text-to-query*, los LLMs suelen emplearse como modelos generativos autorregresivos. Esto significa que generan la salida $y = (y_1, \ldots, y_m)$ un *token* a la vez, donde cada nuevo *token* depende de los generados previamente y de la entrada original. Formalmente, esta generación puede expresarse como:

$$P(y \mid x) \;=\; \prod_{t=1}^{m} P(y_t \mid y_{<t},\, x),$$

donde $P(y_t \mid y_{<t}, x)$ representa la probabilidad del siguiente *token* dado el prefijo ya generado $y_{<t}$ y la entrada x. En la práctica, esta probabilidad se obtiene mediante una capa *softmax* sobre el vocabulario del modelo. A partir de esa distribución, el siguiente *token* puede seleccionarse mediante estrategias como la maximización local o el muestreo.

Esta forma de generación tiene dos consecuencias importantes para *text-to-query*. En primer lugar, el modelo decide cada paso en función de la probabilidad del siguiente *token*, pero no garantiza por sí mismo que la consulta completa sea válida. Por ejemplo, una elección localmente probable puede introducir el nombre de una columna inexistente o una relación incorrecta entre tablas. En segundo lugar, el espacio de salida del modelo está definido por *tokens*, no por las reglas formales del lenguaje de consulta. Por ello, aunque el modelo pueda producir una secuencia que se parezca a una consulta válida, no existe una garantía nativa de que dicha secuencia respete la gramática del lenguaje ni las restricciones del esquema objetivo.

2.1.2. Prompting y aprendizaje en contexto

Sobre esta base de generación, la aplicación a tareas específicas admite dos regímenes complementarios. El *fine-tuning* supervisado modifica los parámetros del modelo mediante pares (pregunta, consulta) con el fin de sesgar $P(y \mid x)$ hacia el lenguaje objetivo. El *aprendizaje en contexto* [Brown et al., 2020], por su parte, mantiene los parámetros congelados y condiciona la generación en un *prompt* que incluye instrucciones y, opcionalmente, ejemplos de la tarea; en este régimen, la distribución efectiva de generación depende de la composición del *prompt*, no de una actualización del modelo.

Ambos regímenes comparten una limitación estructural clave: la salida es el resultado de un muestreo sobre una distribución que desconoce las restricciones formales del lenguaje de consulta y del esquema. Por ello, el estudio de la fidelidad estructural de la salida requiere mecanismos externos al propio LLM.

2.2. Modelos de datos y lenguajes de consulta

Las consultas estudiadas en esta investigación se formulan sobre dos modelos de datos: el modelo relacional y el modelo de grafos de propiedad. Aunque ambos pueden representar información relacionada, lo hacen mediante estructuras distintas y con lenguajes de consulta diferentes. Por ello, esta sección presenta primero cada modelo por separado y luego explica su integración mediante el estándar SQL/PGQ.

2.2.1. Modelo relacional y álgebra relacional

El modelo relacional [Codd, 1970] representa los datos mediante conjuntos de tuplas de aridad fija, donde cada posición se asocia a un atributo con dominio de valores. A su vez un *esquema relacional* $\mathcal{S}_{\text{rel}}$ agrupa estas relaciones y sus restricciones de integridad, como claves primarias, claves foráneas, restricciones de unicidad y dominios válidos.

El álgebra relacional proporciona los operadores formales para consultar este modelo. Entre sus operadores básicos se encuentran la selección σ, la proyección π, la unión natural $\bowtie$, la unión $\cup$, la diferencia $-$ y el renombramiento ρ. Estos operadores son cerrados sobre relaciones, por lo que sus resultados pueden combinarse nuevamente con otros operadores.

Esta base formal es relevante para esta investigación porque permite razonar sobre una consulta antes de ejecutarla. Por ejemplo, es posible verificar si una columna referenciada existe en el esquema, si una operación respeta las relaciones definidas entre tablas o si una composición de operadores es compatible con las restricciones disponibles. De este modo, el modelo relacional no solo define cómo se estructuran los datos, sino también un marco para validar consultas de forma estática.

2.2.2. SQL como lenguaje declarativo

SQL (*Structured Query Language*), estandarizado en la serie ISO/IEC 9075, es el lenguaje declarativo dominante para consultar bases de datos relacionales. Una consulta en forma `SELECT-FROM-WHERE-GROUP BY-HAVING-ORDER BY` se corresponde, con una expresión del álgebra relacional extendida con agregaciones y ordenamiento. La distinción entre la consulta declarativa y el plan físico ejecutado por el motor es relevante para esta tesis: el objetivo de la generación es producir una expresión declarativa bien formada y aterrizada al esquema, en cambio la optimización y ejecución pasan a ser responsabilidad del RDBMS.

2.2.3. Grafos de propiedad y pattern matching

Un *grafo de propiedad* [Angles et al., 2017] se define como una estructura

$$G \;=\; (V,\, E,\, \lambda,\, \mathrm{src},\, \mathrm{tgt},\, \mathrm{prop}),$$

donde V es un conjunto finito de vértices, E un conjunto finito de aristas, $\lambda : V \cup E \to 2^{\Sigma_L}$ asigna a cada elemento un conjunto de etiquetas de un alfabeto, Σ_L, $\mathrm{src}, \mathrm{tgt} : E \to V$ determinan los extremos de cada arista dirigida, y $\mathrm{prop} : (V \cup E) \times K \rightharpoonup D$ asocia a cada elemento y cada clave $k \in K$ un valor parcial en un dominio D.

La forma natural de consultar esta estructura es mediante *pattern matching*. En lugar de describir una consulta como una combinación de tablas, se define un patrón P que debe encontrarse dentro del grafo. Dicho patrón contiene variables, etiquetas y restricciones sobre propiedades. El resultado de la consulta corresponde a las coincidencias entre el patrón y el grafo, es decir, a las asignaciones de sus variables a vértices y aristas concretas que respetan las etiquetas y condiciones declaradas.

Dentro de este tipo de consultas, una clase importante son las *regular path queries* (Regular Path Querys (RPQs)), que permiten describir caminos mediante expresiones regulares sobre las etiquetas de las aristas. Estas consultas generalizan las búsquedas de vecindad, ya que no solo permiten consultar relaciones directas, sino también trayectorias de longitud variable que siguen una secuencia determinada de etiquetas [Angles et al., 2017].

Esta formalización es relevante para la verificación de consultas sobre grafos ya que permiten formular reglas de tipado análogas a las del modelo relacional: toda referencia en una consulta sobre grafos debe corresponder a una etiqueta, un tipo o una propiedad declarada en la especificación de G.

2.2.4. El estándar SQL/PGQ como integración

El estándar ISO/IEC 9075-16:2023 [ISO/IEC JTC 1/SC 32, 2023] introduce SQL/PGQ como una extensión de SQL orientada a la consulta de grafos de propiedad definidos sobre tablas relacionales existentes. Como su construcción principal tenemos la expresión:

```
GRAPH_TABLE(graph MATCH pattern COLUMNS (...))
```

La cual evalua un patrón de grafo y devuelve una relación derivada que puede usarse en conjunto con las cláusulas tradicionales de SQL.

Esta integración resulta relevante dado que une dos formas distintas de representar y consultar datos mantieniendo, por un lado, el modelo relacional (basado en tablas, atributos y restricciones); e integrandolo con el modelo de grafos de propiedad (expresado mediante patrones de vértices, aristas, etiquetas y propiedades).

Esta combinación introduce un reto adicional en el campo investigado ya que no solo se debe generar consultas SQL válidas, sino también patrones de grafo coherentes, así como se debe preservar la consistencia entre ambos niveles. Es por esto que una consulta SQL/PGQ no puede validarse únicamente revisando cada componente por separado, sino que también es necesario verificar que la interacción entre ambos sea estructural y semánticamente correcta.

2.3. Semantic parsing y generación *text-to-query*

2.3.1. *Semantic parsing*: definición y marco general

El *semantic parsing* [Zelle and Mooney, 1996] es la tarea de transformar un enunciado en lenguaje natural u en una representación formal z que pueda ser ejecutada sobre un entorno $\mathcal{E}$. Este entorno puede ser, por ejemplo, una base de datos, un grafo de conocimiento o incluso un sistema físico. El objetivo es que la ejecución de dicha representación formal, denotada como $[\![z]\!]_{\mathcal{E}}$, produzca la respuesta esperada para la pregunta original.

La representación formal z puede adoptar distintas formas, dependiendo del dominio y del tipo de sistema. Por ejemplo, puede tratarse de expresiones en cálculo, lenguajes específicos de dominio, consultas SQL, consultas sobre grafos como *Cypher*, entre otros. Sea cual sea el caso, este problema requiere relacionar tres componentes: el enunciado en lenguaje natural, el entorno sobre el cual se ejecutará la consulta y el mecanismo que traduce el primero en una representación formal válida.

2.3.2. *Text-to-SQL* como caso particular y extensión a *text-to-query*

La tarea *text-to-SQL* es un caso particular de *semantic parsing*. En este caso, la representación formal pertenece al lenguaje SQL y el entorno de ejecución es una base de datos relacional. La entrada del sistema puede expresarse como el par $(u, \mathcal{S}_{\text{rel}})$, donde u es la pregunta en lenguaje natural y $\mathcal{S}_{\text{rel}}$ es el esquema de la base de datos. La salida esperada es una consulta q que, al ejecutarse sobre la base de datos, produzca la respuesta correcta.

La presente tesis aborda una extensión de este problema hacia *text-to-query* sobre SQL/PGQ. En este escenario, la consulta no se limita únicamente a operaciones relacionales, sino que también puede incluir patrones de grafo y combinaciones entre ambos modelos. Por ello, la entrada se define como un par $(u, \mathcal{S})$, donde $\mathcal{S}$ representa un esquema dual:

$$\mathcal{S} \;=\; (\mathcal{S}_{\text{rel}},\, \mathcal{S}_{\text{grafo}}).$$

Aquí, $\mathcal{S}_{\text{rel}}$ corresponde al componente relacional, mientras que $\mathcal{S}_{\text{grafo}}$ describe el componente de grafo de propiedad. La salida esperada es una consulta capaz de combinar operadores SQL tradicionales con patrones de grafo definidos bajo SQL/PGQ.

2.4. Representaciones intermedias

2.4.1. Noción formal de representación intermedia

Una *representación intermedia* (RI) es un lenguaje formal $\mathcal{L}_{\text{RI}}$ dotado de una gramática $\mathcal{G}_{\text{RI}}$, un sistema de tipos $\mathcal{T}_{\text{RI}}$ y una función de compilación

$$\kappa : \mathcal{L}_{\text{RI}} \to \mathcal{L}_{\text{objetivo}}$$

hacia el lenguaje de consulta efectivo. Su rol en un pipeline de generación es mediar entre la salida del componente estadístico y la consulta ejecutable: el modelo produce, o es guiado a producir, instancias bien tipadas de $\mathcal{L}_{\text{RI}}$; el compilador κ genera determinísticamente la consulta final.

2.4.2. Propiedades deseables

Una RI adecuada para *text-to-query* satisface, idealmente, las siguientes propiedades, identificadas a partir de los diseños de RIs previas para parsing semántico [Guo et al., 2019, Gan et al., 2021, Herzig et al., 2021, Nie et al., 2022] y de la formalización de los lenguajes objetivo [Francis et al., 2023, Deutsch et al., 2022]:

Abstracción sintáctica. $\mathcal{L}_{\text{RI}}$ suprime decisiones sintácticas del lenguaje objetivo que no tienen contraparte directa en la formulación en lenguaje natural, tales como alias, el orden de cláusulas, la elección entre `JOIN` y subconsulta, o la presencia explícita de `FROM`, `GROUP BY` y `HAVING`. La motivación sigue la línea de SemQL [Guo et al., 2019] y, más radicalmente, NatSQL [Gan et al., 2021], que prescinden de operadores y cláusulas SQL "raramente encontradas en las descripciones del usuario". Como consecuencia, consultas objetivo semánticamente equivalentes tienden a colapsar en una representación canónica en $\mathcal{L}_{\text{RI}}$, reduciendo el espacio de salida que el modelo debe predecir.

Tipado composicional. La buena formación de una instancia se decide mediante un sistema de tipos estático: cada constructor declara los tipos de sus operandos y el tipo del término resultante se deriva de los de sus subtérminos. Esta propiedad sigue la tradición de los cálculos formales para lenguajes de consulta de grafos, en particular GPC [Francis et al., 2023], donde un conjunto de reglas de tipado garantiza que

toda expresión bien tipada admite un esquema $\mathrm{sch}(\xi)$ derivable composicionalmente. El tipado opera puramente sobre la estructura de la RI y el esquema de la base de datos, sin requerir evaluación.

Compilación dirigida por sintaxis. La función de compilación $\kappa : \mathcal{L}_{\mathrm{RI}} \times \mathcal{S} \to \mathcal{L}_{\mathrm{obj}}$, donde $\mathcal{S}$ es el esquema, es total sobre instancias bien tipadas y se define por inducción estructural sobre la RI. La indeterminación propia de la generación neuronal queda así confinada al espacio $\mathcal{L}_{\mathrm{RI}}$, en línea con el principio de *deterministic inference* de IRNet [Guo et al., 2019] y con la separación entre parser semántico y compilador *source-to-source* adoptada por GraphQ IR [Nie et al., 2022]. La dependencia explícita del esquema es necesaria porque parte de la información elidida en la RI (e.g. inferencia de `GROUP BY`, resolución de claves foráneas para `JOIN`) se reconstruye consultando $\mathcal{S}$.

Expresividad adecuada. $\mathcal{L}_{\mathrm{RI}}$ debe cubrir las construcciones del lenguaje objetivo que aparecen en la distribución de tareas considerada. Para SQL/PGQ [Deutsch et al., 2022], esto exige soporte simultáneo para las tres capas conceptuales del estándar [Bonifati et al., 2025]: (i) emparejamiento de patrones de grafo, (ii) operadores del álgebra relacional, y (iii) composición híbrida mediante `GRAPH_TABLE`. Las RIs existentes cubren parcialmente alguno de estos fragmentos, como podrían ser SemQL/NatSQL para SQL relacional [Guo et al., 2019, Gan et al., 2021]y GraphQ IR para Cypher/SPARQL/KoPL [Nie et al., 2022]; pero, hasta donde pudo verificarse, no existe una RI que cubra el fragmento híbrido relacional-grafo de SQL/PGQ.

2.5. Verificación de consultas generadas

Si bien el objetivo es la producción de una cadena de consulta, el proceso de *text-to-query* no termina ahí, ya que es necesario evaluar si la consulta generada es consistente con el esquema, si puede ejecutarse correctamente y si su comportamiento es coherente frente a la pregunta original, a este proceso le denominamos verificación y en esta sección detallaremos tres niveles y como se aplican al problema investigado.

2.5.1. Verificación estática

La verificación estática designa el análisis de una consulta, o de su RI, sin ejecutarla, mediante la aplicación de reglas sobre su estructura y el esquema. En el contexto de *text-to-query*, los chequeos estáticos relevantes incluyen: existencia de tablas, columnas, etiquetas y propiedades referenciadas; consistencia de tipos en operadores y condiciones; satisfacibilidad trivial de restricciones, como detección de condiciones contradictorias evidentes; y, en consultas híbridas, coherencia entre los componentes relacional y de grafo.

Formalmente, la verificación estática puede modelarse como un conjunto de predicados $\Phi = \{\varphi_1, \ldots, \varphi_k\}$ sobre consultas y esquemas. Una consulta q se considera *estáticamente válida* respecto a un esquema $\mathcal{S}$ si se cumple $\bigwedge_{i=1}^{k} \varphi_i(q, \mathcal{S})$. La decidibilidad local de cada φ_i hace de la verificación estática un mecanismo de costo constante o lineal respecto al tamaño de q, y por ello resulta aplicable en un bucle de refinamiento sin comprometer la eficiencia global.

2.5.2. Verificación dinámica

La verificación dinámica ejecuta la consulta en un entorno controlado y analiza la respuesta del motor. El motor actúa como *oráculo parcial*: distingue consultas sintáctica y referencialmente válidas de aquellas que disparan errores de ejecución, pero no decide, en general, la corrección semántica de una consulta ejecutable. Las señales útiles en este régimen incluyen los errores tipados del motor, la cardinalidad anómala del resultado, entendida como un resultado vacío cuando se espera contenido o viceversa, la presencia inesperada de valores nulos y las desviaciones respecto a respuestas de referencia cuando estas existen.

2.5.3. Testing metamórfico

El *testing metamórfico* [Chen et al., 1998] extiende la verificación dinámica más allá de lo explícito. La idea central consiste en identificar relaciones entre entradas y salidas que deben preservarse bajo transformaciones conocidas, denominadas *relaciones metamórficas*. En *text-to-query*, relaciones como la invariancia del resultado bajo renombramiento de alias, la inclusión del resultado de una consulta con `LIMIT` en el de la misma consulta sin dicha cláusula, o la preservación de cardinalidades bajo reformulaciones semánticamente equivalentes, ofrecen un mecanismo para detectar inconsistencias sin conocer de antemano la respuesta correcta. La violación de una relación metamórfica evidencia un error; su satisfacción, sin embargo, no certifica corrección, sino que acota el espacio de hipótesis de error.

Capítulo 3

Arquitecturas y Enfoques Actuales en Text-to-Query

El capítulo anterior presentó los fundamentos necesarios para comprender la generación *text-to-query*: los modelos de lenguaje, las limitaciones de estos, las representaciones intermedias, los mecanismos de verificación y el estándar híbrido SQL/PGQ. A partir de esa base, este capítulo revisa los trabajos actuales en sistemas *text-to-query*.

Esta revisión no se plantea como una lista cronológica de trabajos, sino como un análisis organizado por dimensiones, decisión tomada dado que la literatura reciente aborda el problema desde criterios diversos: algunas revisiones clasifican los sistemas según la arquitectura del modelo, otras según las etapa del *pipeline*, e incluso según la estrategia de *prompting* empleada [Quamar et al., 2022], entre otros. Aunque estas clasificaciones son útiles, no siempre permiten comparar los trabajos bajo criterios comunes ni identificar con claridad las brechas de investigación.

Por ello, este capítulo adopta una taxonomía multidimensional para analizar la literatura de manera más ordenada.

3.1. Marco taxonómico del análisis

Resolver el problema *text-to-query* obliga a considerar simultáneamente decisiones de: aprendizaje, orquestación arquitectónica, representación, enriquecimiento contextual y criterios de verificación. Estas decisiones operan en diferentes niveles y etapas, incluyendo combinaciones muy variadas en la literatura [Hong et al., 2024, Shi et al., 2025, Liu et al., 2025a].

En base a esta complejidad y vacío identificado, se desarrolló una taxonomía con seis dimensiones complementarias para clasificar los sistemas *text-to-query*. La taxonomía presentada fue elaborada recogiendo y reorganizando ejes ya presentes en surveys recientes [Hong et al., 2024, Liu et al., 2024, Shi et al., 2025, Qin et al., 2022], y añadiendo dos contribuciones específicas, producto de la revisión de literatura y de los patrones observados: separar el *paradigma de razonamiento* y las *propiedades de evaluación* como dimensiones

independientes, de forma tal que un mismo trabajo pueda recibir etiquetas en cada eje sin que las etiquetas de uno predetermine las del otro.

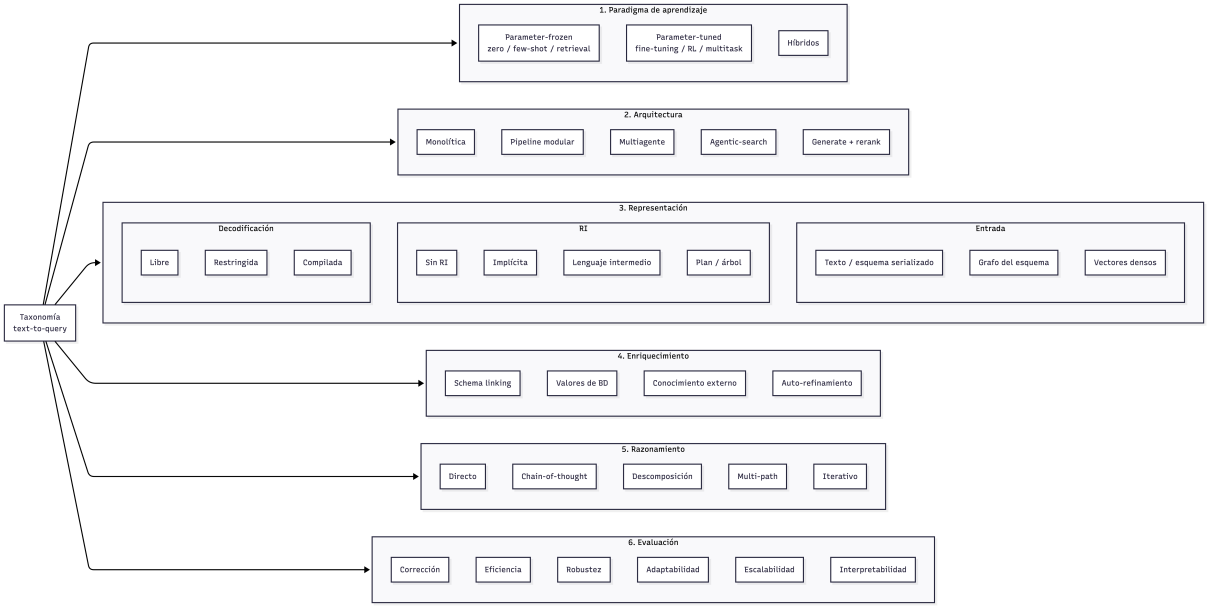

Figura 3.1: Taxonomía multidimensional propuesta. Cada uno de los seis ejes ortogonales agrupa las decisiones de diseño observadas en la literatura de *text-to-query* y sus principales subcategorías.

La Figura 3.1 sintetiza las seis dimensiones consideradas, cuya descripción y justificación se resumen a continuación:

1. **Paradigma de aprendizaje:** Abarca cómo se adapta el modelo base a la tarea, ya sea con parámetros congelados (*zero-shot*, *few-shot* estático o con recuperación dinámica), parámetros ajustados (*fine-tuning*, preentrenamiento continuo, aprendizaje multitarea o por refuerzo) o estrategias híbridas. Esta dimensión ayuda a determinar el costo de adaptación, la dependencia de datos anotados y la complejidad de transferencia a nuevos dominios.

2. **Arquitectura del sistema:** La distribución de la ejecución entre el modelo y los diferentes módulos auxiliares, distinguiendo configuraciones monolíticas, *pipelines* modulares, multiagente, *agentic-search* y generación con reordenamiento.

3. **Representación:** Las estructuras que median entre la pregunta y la consulta final, considerando codificación de la entrada, representación intermedia y estrategia de decodificación.

4. **Mecanismos de enriquecimiento contextual:** La información adicional que ancla la salida al esquema y al dominio, incluyendo *schema linking*, recuperación de contenido de la base de datos, conocimiento externo o ciclos de auto-refinamiento. Se trata como dimensión propia porque su efecto sobre la corrección actúa de forma transversal a la arquitectura y al paradigma de aprendizaje.

5. **Paradigma de razonamiento:** Cómo se estructura la inferencia que lleva a la consulta, en variantes directa, en cadena de pensamiento, descompositiva, de múltiples caminos o de retroalimentación iterativa.

6. **Evaluación:** Las propiedades del sistema que prioriza explícitamente cada trabajo, entre corrección, eficiencia, robustez, adaptabilidad, escalabilidad e interpretabilidad. Se incorpora como eje independiente para hacer visible el sesgo dominante hacia la corrección y permitir comparaciones sobre propiedades críticas en consultas empresariales.

A partir de esta categorización, las siguientes secciones revisarán la literatura reciente analizando, por dimensión, cómo se han posicionado los trabajos más representativos, enfocándose en propuestas publicadas mayoritariamente entre 2017 y 2026, abarcando de esta forma, desde los primeros sistemas neuronales basados en codificadores especializados hasta las arquitecturas multiagente más recientes.

3.2. Paradigma de aprendizaje

Esta sección analiza la forma en que los sistemas *text-to-query* adaptan su modelo base a la tarea, ya que si bien la mayoría de los trabajos revisados utiliza LLMs o modelos preentrenados de tipo *encoder-decoder*, no todos los sistemas los emplean de la misma manera.

En términos generales, pueden distinguirse tres estrategias principales: La primera consiste en mantener los parámetros del modelo congelados y adaptar su comportamiento únicamente mediante el *prompt*. La segunda ajusta los parámetros del modelo mediante *fine-tuning* u otros procesos de entrenamiento adicional. La tercera combina ambas aproximaciones, por ejemplo, usando modelos congelados para ciertas etapas del *pipeline* y componentes ajustados para tareas específicas como selección, reordenamiento o verificación.

Esta distinción es relevante porque cada estrategia implica compromisos distintos en términos de costo de adaptación, necesidad de datos anotados, flexibilidad frente a nuevos dominios y control sobre el comportamiento del sistema.

3.2.1. Aprendizaje con parámetros congelados

En esta categoría, la adaptación a la tarea se realiza únicamente mediante el diseño del *prompt*, sin modificar los parámetros internos del modelo. Por tanto, la diferencia principal entre los enfoques no está en el entrenamiento del modelo, sino en la forma en que se construye el contexto que se entrega para generar la consulta.

(a) *Zero-shot.* Los enfoques *zero-shot* prescinden de ejemplos demostrativos y se apoyan solo en la pregunta del usuario, la descripción del esquema y las instrucciones incluidas en el *prompt*. C3-SQL [X. Dong and et al., 2023] es representativo de esta línea, ya que muestra que un *prompt* cuidadosamente diseñado puede alcanzar resultados competitivos sobre Spider sin incorporar ejemplos previos. De manera similar, Alpha-SQL [B. Li and Luo, 2025], aunque corresponde a una arquitectura agéntica, puede considerarse *zero-shot*

porque no depende de demostraciones específicas del dominio; en su lugar, explora el espacio de consultas mediante búsqueda guiada sobre *prompts* dinámicos.

La principal ventaja de esta subcategoría es su bajo costo de adaptación, pues no requiere datos anotados ni selección de ejemplos. Sin embargo, su desempeño depende fuertemente de la capacidad del modelo base y tiende a degradarse cuando el esquema es grande, poco convencional o presenta relaciones difíciles de inferir solo a partir de su descripción.

(b) *Few-shot in-context learning.* La mayoría de los sistemas recientes basados en LLMs se ubica en esta subcategoría. En estos enfoques, el modelo recibe un conjunto de ejemplos de pares pregunta-consulta que ilustran el comportamiento esperado. Estos ejemplos pueden ayudar al modelo a reconocer patrones de formulación, estructuras frecuentes de consulta y formas adecuadas de vincular la pregunta con el esquema.

DIN-SQL [Pourreza and Rafiei, 2023] es uno de los trabajos influyentes en esta línea. Su propuesta descompone la tarea en subpasos y diseña *prompts* específicos para cada uno, apoyados en ejemplos seleccionados manualmente. Enfoques posteriores o relacionados, como DAIL-SQL [D. Gao and et al., 2023], ACT-SQL [Zhang and et al., 2023], PET-SQL [Li and et al., 2024], DEA-SQL [Xie and et al., 2024], RSL-SQL [et al., 2024], CHESS [S. Talaei and et al., 2024], MAC-SQL [B. Wang and et al., 2023], TA-SQL [Qu et al., 2024], CoE-SQL [H. Zhang and et al., 2024], PURPLE [T. Ren and et al., 2024], ODIS [Chang and Fosler-Lussier, 2023], Gen-SQL [J. Shi and et al., 2025], SGU-SQL [Qinggang Zhang, 2025], CRED-SQL [Shaoming Duan, 2025] y CRUSH4SQL [et al., 2023], entre otros, mantienen esta lógica general, aunque la combinan con distintas formas de descomposición, recuperación, verificación o refinamiento.

La amplitud de trabajos en esta subcategoría muestra la madurez del paradigma de aprendizaje en contexto. No obstante, también evidencia una limitación: muchas mejoras dependen de ajustes en la ingeniería de *prompts*, la selección de ejemplos o la organización del contexto, más que de cambios profundos en la arquitectura del sistema.

(c) Aprendizaje en contexto dinámico con recuperación. Una variante más flexible del *few-shot* consiste en seleccionar dinámicamente los ejemplos que se incluyen en el *prompt*. En lugar de usar un conjunto fijo de demostraciones, el sistema recupera ejemplos similares a la pregunta de entrada o al esquema objetivo. Esta estrategia busca que el contexto proporcionado al modelo sea más relevante para cada caso particular.

OpenSearch-SQL [X. Xie and Guo, 2025] ilustra esta aproximación mediante un módulo dedicado a la recuperación de ejemplos. CRED-SQL [Shaoming Duan, 2025] también contempla esta estrategia como una extensión posible dentro de su arquitectura. La ventaja principal frente al *few-shot* estático es su mayor adaptabilidad a dominios heterogéneos. Sin embargo, esta adaptabilidad depende de la calidad, cobertura y organización del corpus de ejemplos disponible. Si los ejemplos recuperados son poco representativos o introducen patrones incorrectos, el desempeño del modelo puede verse afectado.

3.2.2. Aprendizaje con parámetros ajustados

En esta categoría se modifican los parámetros del modelo mediante entrenamiento adicional sobre datos específicos de la tarea. Es la familia históricamente más extendida en *text-to-SQL* previa al auge de los LLMs de propósito general.

(a) Fine-tuning supervisado. Trabajos como Seq2SQL [Victor Zhong, 2017], BRIDGE [Lin and et al., 2020], RAT-SQL [B. Wang and et al., 2021], SmBoP [Rubin and Berant, 2021], RaSaP [J. Huang and et al., 2021], SHiP [L. Dou and et al., 2022], RASAT [J. Qi and et al., 2022], PICARD [Scholak and et al., 2022], S^2SQL [Hui and et al., 2022], Graphix-T5 [Li and et al., 2022], RESDSQL [H. Li and Chen, 2023], T5-3B+NatSQL [Gan and et al., 2023], SQLFormer [A. Bazaga and et al., 2023], CatSQL [H. Fu and et al., 2023] y DTS-SQL [Pourreza and Rafiei, 2024] ajustan modelos completos sobre conjuntos como Spider o WikiSQL. Las arquitecturas base varían desde redes específicas para *text-to-SQL* (Seq2SQL, BRIDGE, RAT-SQL, SmBoP) hasta variantes de T5 adaptadas (PICARD, RESDSQL, Graphix-T5). Los sistemas de esta sub-categoría dominaron los rankings de los benchmarks controlados durante el período pre-LLM (2017–2022); su costo de entrenamiento y su dependencia del dominio de entrenamiento limitan, sin embargo, su transferencia directa a entornos empresariales.

(b) Preentrenamiento continuo. CodeS [H. Li and et al., 2024] representa una variante intermedia: parte de un modelo de código preentrenado de propósito general y aplica preentrenamiento continuo sobre un corpus masivo orientado a SQL antes del *fine-tuning* supervisado final. La estrategia mejora la transferencia a dominios desconocidos, pero requiere infraestructura considerable y un corpus de entrenamiento curado.

(c) Aprendizaje multi-tarea. Algunos trabajos entrenan el modelo sobre varias tareas relacionadas para mejorar la generalización. TKK [C. Gao and et al., 2022] aborda *text-to-SQL* junto con tareas auxiliares de comprensión de esquema. ROUTE [Y. Qin and et al., 2025] adopta el aprendizaje multi-tarea como principio organizador del *fine-tuning*, integrando tareas de generación, corrección y razonamiento sobre esquemas. PARSQL [Yaxun Dai, 2025] sigue una línea similar al combinar generación con selección entre candidatos. La motivación común es que las tareas auxiliares fuerzan al modelo a desarrollar representaciones internas más robustas del esquema y de la sintaxis SQL.

(d) Aprendizaje por refuerzo. STRuCT-LLM [Stoisser et al., 2025] incorpora aprendizaje por refuerzo para alinear la generación con señales de ejecución y con tareas de razonamiento sobre tablas y grafos simultáneamente. Seq2SQL [Victor Zhong, 2017], en una época mucho más temprana, ya empleaba aprendizaje por refuerzo basado en la recompensa de ejecución. La adopción del paradigma sigue siendo minoritaria pero ha resurgido con fuerza en propuestas recientes orientadas a unificar razonamiento estructurado con generación.

3.2.3. Estrategias híbridas

Una fracción creciente de los sistemas más recientes combina componentes con parámetros congelados y componentes ajustados. XiYan-SQL [Yingqi Gao, 2024] adopta una arquitectura multiagente en la que un generador puede ser un LLM ajustado de la familia XiYanSQL-QwenCoder mientras que módulos auxiliares operan *in-context*. CHASE-SQL [M. Pourreza and et al., 2025] combina generadores *in-context* con un seleccionador entrenado mediante *fine-tuning*. ZeroNL2SQL [Z. Gu and et al., 2024] distribuye la carga entre un modelo preentrenado pequeño y un LLM congelado. MetaSQL [Y. Fan and et al., 2024] aplica un esquema análogo en la fase de selección sobre múltiples candidatos. La motivación es pragmática: aprovechar la capacidad de generalización de los LLMs sin renunciar al control fino que provee el ajuste sobre los componentes críticos.

3.2.4. Observaciones críticas

La revisión de esta dimensión muestra un desplazamiento progresivo del *fine-tuning* de extremo a extremo hacia configuraciones híbridas y *in-context*. Esta tendencia es coherente con el aumento del costo computacional asociado al entrenamiento de LLMs de gran tamaño. Sin embargo, los sistemas puramente *in-context* acumulan dos limitaciones recurrentes: dependencia fuerte del modelo subyacente, lo que dificulta auditar el sistema desde el punto de vista de gobernanza, y ausencia de mecanismos formales para garantizar que la salida respete el esquema. La investigación de la presente tesis se posiciona como agnóstica al paradigma de aprendizaje subyacente: el énfasis se traslada a la representación intermedia y al bucle de verificación, de modo que la corrección estructural no descanse exclusivamente en la calidad estadística del generador.

3.3. Arquitectura del Sistema

Esta sección describe cómo se organiza el cómputo en los sistemas *text-to-query*. La distinción es independiente del paradigma de aprendizaje: un mismo paradigma puede dar lugar a arquitecturas monolíticas, modulares, multiagente, agénticas o de generación con reordenamiento.

3.3.1. Arquitecturas monolíticas

En las arquitecturas monolíticas, una sola invocación del modelo produce la consulta completa. Es la configuración natural de los sistemas *end-to-end* entrenados para mapear pregunta a consulta directamente. Pertenecen a esta categoría Seq2SQL [Victor Zhong, 2017], BRIDGE [Lin and et al., 2020], RAT-SQL [B. Wang and et al., 2021], SmBoP [Rubin and Berant, 2021], RaSaP [J. Huang and et al., 2021], RASAT [J. Qi and et al., 2022], PICARD [Scholak and et al., 2022], S^2SQL [Hui and et al., 2022], Graphix-T5 [Li and et al., 2022], T5-3B+NatSQL [Gan and et al., 2023], SQLFormer [A. Bazaga and et al., 2023], CatSQL [H. Fu and et al., 2023], SHiP [L. Dou and et al., 2022], CodeS [H. Li and

et al., 2024], DAIL-SQL [D. Gao and et al., 2023], ACT-SQL [Zhang and et al., 2023], ODIS [Chang and Fosler-Lussier, 2023], SGU-SQL [Qinggang Zhang, 2025] y STRuCT-LLM [Stoisser et al., 2025]. Aunque algunos de estos sistemas internamente realizan operaciones de codificación o decodificación sofisticadas, conceptualmente la generación de la consulta es una sola inferencia. La ventaja principal de esta organización es la simplicidad operativa; la desventaja es que cualquier error se propaga sin posibilidad de inspección intermedia ni de intervención modular.

3.3.2. Pipelines modulares

En esta categoría, la generación se distribuye entre etapas secuenciales con responsabilidades distintas: típicamente *schema linking*, generación, validación y refinamiento. DIN-SQL [Pourreza and Rafiei, 2023] popularizó una variante influyente del enfoque, descomponiendo el proceso en cuatro sub-tareas. La línea ha sido seguida por numerosos trabajos: RESDSQL [H. Li and Chen, 2023], DTS-SQL [Pourreza and Rafiei, 2024], PURPLE [T. Ren and et al., 2024], RSL-SQL [et al., 2024], DEA-SQL [Xie and et al., 2024], CoE-SQL [H. Zhang and et al., 2024], PET-SQL [Li and et al., 2024], SuperSQL [B. Li and et al., 2024], TA-SQL [Qu et al., 2024], FinSQL [Zhang and et al., 2024], TKK [C. Gao and et al., 2022], ZeroNL2SQL [Z. Gu and et al., 2024], SteinerSQL [Xutao Mao, 2025], Gen-SQL [J. Shi and et al., 2025], BLENDSQL [et al., 2025], CRED-SQL [Shaoming Duan, 2025], CRUSH4SQL [et al., 2023], SelECT-SQL [Z. Gu and et al., 2023], CHESS [S. Talaei and et al., 2024], C3-SQL [X. Dong and et al., 2023] y Wren AI [Chi, 2025]. Los *pipelines* modulares facilitan la inspección, el reemplazo de componentes y la integración con sistemas de verificación, lo que los hace especialmente atractivos para entornos empresariales. Su limitación principal es la rigidez del orden de ejecución y la posible amplificación de errores entre etapas.

3.3.3. Arquitecturas multiagente

Una variante reciente del enfoque modular dota a cada componente de autonomía e iniciativa, configurándolo como un agente que se comunica con otros mediante mensajes. MAC-SQL [B. Wang and et al., 2023] es uno de los trabajos influyentes en consolidar esta organización para *text-to-SQL*, con agentes especializados en *schema linking*, descomposición y refinamiento. CHESS [S. Talaei and et al., 2024] extiende esta línea con cuatro agentes —recuperación de información, selección de esquema, generación y verificación— e incorpora un agente *Unit Tester* que actúa como verificador semántico. OpenSearch-SQL [X. Xie and Guo, 2025] y TS-SQL [Wenbo Xu, 2025] adoptan también una organización multiagente. XiYan-SQL [Yingqi Gao, 2024] combina varios agentes generadores con un agente seleccionador. La diferencia operativa entre un *pipeline* modular y un sistema multiagente es a menudo gradual; en la práctica, lo distintivo del enfoque multiagente es la capacidad de varios componentes de invocar al modelo de forma asíncrona o cíclica.

3.3.4. Arquitecturas agentic-search

Algunos trabajos abandonan la idea de un orden fijo y adoptan en su lugar una exploración guiada del espacio de soluciones. Alpha-SQL [B. Li and Luo, 2025] ilustra esta categoría: utiliza un modelo para proponer acciones (refinar el esquema, generar un fragmento, verificar una sub-consulta) y un mecanismo de búsqueda para decidir qué acción ejecutar a continuación. CHASE-SQL [M. Pourreza and et al., 2025] también incorpora elementos de exploración, aunque combinados con generación múltiple. La ventaja del enfoque es su flexibilidad ante consultas complejas; su desventaja, el costo computacional asociado a la búsqueda y la dificultad de garantizar terminación.

3.3.5. Arquitecturas de generación con reordenamiento

En esta categoría, el sistema produce varios candidatos de consulta y un módulo posterior selecciona el mejor. CHASE-SQL [M. Pourreza and et al., 2025], XiYan-SQL [Yingqi Gao, 2024] y MetaSQL [Y. Fan and et al., 2024] adoptan explícitamente esta estrategia. PARSQL [Yaxun Dai, 2025] también opera sobre un conjunto de candidatos y G^3R [Y. Xiang and et al., 2023] incorpora una etapa de reordenamiento basada en alineación. Trabajos clásicos como N-best List Rerankers [L. Zeng and Hakkani-Tur, 2022] sentaron las bases del enfoque. La motivación es robusta: si el generador produce un candidato correcto entre varios, un reordenador entrenado puede recuperarlo aún cuando no sea el más probable a priori. La debilidad es la dependencia de la diversidad real del conjunto generado.

3.3.6. Observaciones críticas

La distribución de los trabajos analizados por arquitectura muestra una migración progresiva desde sistemas monolíticos —propios de la era *pre-LLM*— hacia *pipelines* modulares, multiagente y agénticos. Esta evolución responde tanto a la creciente complejidad de las consultas objetivo como a la necesidad de inspeccionar y verificar la salida en entornos críticos. Sin embargo, una limitación común a las arquitecturas modulares y multiagente es que la verificación semántica, cuando existe, suele ejecutarse al final del proceso sobre la consulta ya generada, y rara vez se realiza sobre una representación intermedia que admita análisis estático. Esta brecha es uno de los puntos que la propuesta de la presente tesis aborda explícitamente.

3.4. Estrategias de Representación

La forma en que un sistema representa la pregunta, el esquema y la consulta en formación condiciona qué tipos de errores puede detectar y qué tipos de control puede ejercer sobre la salida. Esta sección agrupa los trabajos según los tres ejes complementarios de la dimensión: codificación de la entrada, representación intermedia entre comprensión y generación, y estrategia de decodificación de la consulta final.

3.4.1. Codificación de la entrada

(a) Esquema serializado en lenguaje natural. La práctica dominante en sistemas basados en LLMs consiste en serializar el esquema como un fragmento de texto que se concatena con la pregunta. DIN-SQL [Pourreza and Rafiei, 2023], DAIL-SQL [D. Gao and et al., 2023], ACT-SQL [Zhang and et al., 2023], MAC-SQL [B. Wang and et al., 2023], CHESS [S. Talaei and et al., 2024] y la mayoría de los sistemas *in-context* recientes emplean esta codificación. Su ventaja es la simplicidad y la compatibilidad universal con LLMs; su desventaja es que se pierden las relaciones estructurales del esquema, salvo que se reintroduzcan explícitamente en el texto.

(b) Grafos del esquema. Una línea de trabajos modela explícitamente el esquema como un grafo, donde los nodos representan tablas y columnas y las aristas representan claves foráneas y otras relaciones. RAT-SQL [B. Wang and et al., 2021], S^2SQL [Hui and et al., 2022] y Graphix-T5 [Li and et al., 2022] incorporan capas de atención sensibles a la estructura del grafo. La motivación es facilitar el razonamiento sobre uniones complejas y rutas de *join* no triviales. El costo es una mayor complejidad arquitectónica y la necesidad de construir el grafo del esquema como paso de preprocesamiento.

(c) Esquema enriquecido (M-Schema y similares). XiYan-SQL [Yingqi Gao, 2024] introduce M-Schema, un formato que combina el esquema con metadatos y ejemplos de valores en una representación textual estructurada que el LLM consume directamente. Esta codificación intermedia entre texto plano y grafo busca preservar la simplicidad del primero sin perder la información estructural del segundo. Otros trabajos, como CHESS [S. Talaei and et al., 2024] y PET-SQL [Li and et al., 2024], adoptan formatos análogos en su etapa de preparación del *prompt*.

(d) Recuperación previa al modelo. En esquemas masivos, no es viable serializar el esquema completo en el *prompt*. CRUSH4SQL [et al., 2023] aborda esta restricción mediante recuperación: un módulo previo selecciona los elementos del esquema relevantes para la pregunta antes de invocar al modelo. TS-SQL [Wenbo Xu, 2025] y SuperSQL [B. Li and et al., 2024] contemplan estrategias análogas. El compromiso es entre cobertura —no omitir elementos críticos— y compacidad —no saturar el contexto.

3.4.2. Representación intermedia

Esta sub-dimensión es central para la presente tesis. Se distinguen seis sub-categorías según el tipo de RI que cada sistema utiliza, si es que utiliza alguna.

(a) Sin RI explícita. La consulta se genera directamente como secuencia de tokens del lenguaje destino. Pertenecen a esta categoría Seq2SQL [Victor Zhong, 2017], BRIDGE [Lin and et al., 2020], DAIL-SQL [D. Gao and et al., 2023], ACT-SQL [Zhang and et al., 2023], ODIS [Chang and Fosler-Lussier, 2023], SuperSQL [B. Li and et al., 2024] y la mayoría

de los sistemas *in-context* basados en *prompting*. La ausencia de RI explícita simplifica la arquitectura, pero traslada toda la carga de corrección estructural al generador.

(b) Esquema como RI implícita. Algunos trabajos no introducen una RI dedicada, pero usan el esquema enriquecido como sustrato sobre el cual se realiza el razonamiento previo a la generación. Es el caso de RAT-SQL [B. Wang and et al., 2021] y, en parte, de XiYan-SQL [Yingqi Gao, 2024] con su M-Schema. La consulta sigue siendo SQL nativo, pero la representación del esquema actúa como puente estructurado.

(c) Lenguaje intermedio dedicado. La línea más explícita hacia una RI propiamente dicha la inicia IRNet con SemQL [Guo et al., 2019], un lenguaje intermedio que separa interpretación semántica de sintaxis SQL concreta. NatSQL [Gan et al., 2021] sigue una motivación análoga: define un lenguaje SQL simplificado sobre el que se entrena el modelo, y un compilador trivial lo convierte a SQL ejecutable. El trabajo T5-3B+NatSQL [Gan and et al., 2023] se apoya directamente sobre esta RI para mejorar la generalización. CatSQL [H. Fu and et al., 2023] y ZeroNL2SQL [Z. Gu and et al., 2024] contemplan formas similares de representación intermedia. La ventaja estructural es que la RI restringe el espacio de errores posibles del modelo; la limitación es que la RI suele estar acoplada a un dialecto específico de SQL.

(d) Plan estructurado. Algunos sistemas representan la consulta como un plan de ejecución o como una secuencia de operaciones declarativas. SQLFormer [A. Bazaga and et al., 2023] sigue parcialmente esta línea al organizar la generación alrededor de operaciones primitivas. SteinerSQL [Xutao Mao, 2025] representa la consulta como un árbol sobre el grafo del esquema, lo que se aproxima a una representación plan-céntrica. La ventaja es la trazabilidad del razonamiento; la limitación es el costo de adoptar y mantener un formalismo propio.

(e) Árbol abstracto. SmBoP [Rubin and Berant, 2021] construye la consulta de forma composicional como un árbol abstracto sobre operadores relacionales, generando *bottom-up*. PICARD [Scholak and et al., 2022] no expone explícitamente un árbol como artefacto, pero su decodificación restringida opera implícitamente sobre la gramática del lenguaje, lo que lo posiciona en una categoría afín. RaSaP [J. Huang and et al., 2021] y SQLFormer [A. Bazaga and et al., 2023] también incorporan estructura arbórea en distintos grados.

(f) Descomposición declarativa. DIN-SQL [Pourreza and Rafiei, 2023] introduce una forma específica de RI implícita: la pregunta se descompone en sub-preguntas y cada sub-pregunta produce una sub-consulta, que luego se ensambla. La descomposición no es una RI estructural, pero juega un papel análogo al distribuir el razonamiento. CHESS [S. Talaei and et al., 2024] y MAC-SQL [B. Wang and et al., 2023] adoptan estrategias similares.

3.4.3. Estrategias de decodificación

(a) Decodificación libre. El modelo genera la consulta como texto sin restricciones formales. Es la estrategia dominante en sistemas *in-context*: DIN-SQL [Pourreza and Rafiei, 2023], DAIL-SQL [D. Gao and et al., 2023], ACT-SQL [Zhang and et al., 2023], CHESS [S. Talaei and et al., 2024], MAC-SQL [B. Wang and et al., 2023], OpenSearch-SQL [X. Xie and Guo, 2025], XiYan-SQL [Yingqi Gao, 2024], entre otros. Su ventaja es la flexibilidad; su desventaja, la posibilidad de generar consultas sintácticamente inválidas.

(b) Decodificación restringida por gramática. PICARD [Scholak and et al., 2022] establece el patrón de referencia de esta categoría: integra una gramática de SQL en el decodificador para descartar en tiempo real candidatos que violen el lenguaje. La técnica elimina por completo los errores sintácticos al costo de una decodificación más cara. PICARD inspira aplicaciones posteriores en otros componentes de *pipelines* modulares.

(c) Decodificación composicional. SmBoP [Rubin and Berant, 2021] genera la consulta de manera *bottom-up* sobre operadores relacionales, imponiendo estructura desde el inicio de la derivación. La estrategia es más compleja de implementar pero garantiza por construcción ciertos invariantes estructurales.

(d) Compilación desde la representación intermedia. Cuando el sistema utiliza una RI explícita, la decodificación final puede reducirse a la compilación determinista de la RI al lenguaje destino. IRNet [Guo et al., 2019] y NatSQL [Gan et al., 2021] operan así: el modelo genera la RI y un componente determinista produce la consulta. Esta estrategia separa por completo el riesgo estadístico del riesgo formal.

3.4.4. Observaciones críticas

La revisión muestra que la mayoría de los sistemas recientes basados en LLMs ha priorizado simplificar la representación —esquema serializado, decodificación libre, sin RI explícita— delegando la corrección al generador. Esta decisión ha sido razonable mientras los benchmarks dominantes contenían esquemas de complejidad moderada y consultas SQL relativamente regulares. Sin embargo, en escenarios empresariales y, especialmente, en consultas híbridas sobre el estándar SQL/PGQ, la ausencia de RI explícita amplifica la propagación de alucinaciones estructurales: no existe una capa intermedia donde verificar la consistencia de tipos, la validez de las uniones o la coherencia entre los componentes relacionales y de patrón de grafo. La investigación de esta tesis se posiciona deliberadamente en la sub-categoría de RI explícita con compilación determinista, ampliada para cubrir tanto SQL como PGQ y enriquecida con verificación estática sobre la propia RI.

3.5. Mecanismos de Enriquecimiento Contextual

Esta sección agrupa las técnicas que aportan información adicional al modelo más allá de la pregunta y el esquema básico. Los cuatro mecanismos no son excluyentes: muchos sistemas combinan varios.

3.5.1. Schema linking

El *schema linking* consiste en establecer correspondencias explícitas entre menciones del texto y elementos del esquema (tablas, columnas, valores). Esta operación es central en *text-to-SQL* y ha recibido tratamiento dedicado tanto como módulo independiente como integrado dentro de la arquitectura. RAT-SQL [B. Wang and et al., 2021] la incorpora mediante atención sensible a relaciones del esquema. RESDSQL [H. Li and Chen, 2023], DTS-SQL [Pourreza and Rafiei, 2024], DIN-SQL [Pourreza and Rafiei, 2023], DEA-SQL [Xie and et al., 2024], RSL-SQL [et al., 2024], CHESS [S. Talaei and et al., 2024], MAC-SQL [B. Wang and et al., 2023], TA-SQL [Qu et al., 2024], PURPLE [T. Ren and et al., 2024], TS-SQL [Wenbo Xu, 2025] y CRUSH4SQL [et al., 2023], entre otros, dedican un módulo o etapa específica al *schema linking*. La calidad de esta etapa se correlaciona fuertemente con la corrección final del sistema, lo que explica la atención sostenida que ha recibido.

3.5.2. Recuperación de contenido de la base de datos

Algunos sistemas incorporan valores reales de las columnas para resolver ambigüedades léxicas que el esquema solo no permite resolver —por ejemplo, distinguir entre dos columnas con nombres similares pero contenido distinto. BRIDGE [Lin and et al., 2020] fue uno de los primeros en hacerlo de forma sistemática. Trabajos posteriores como CHESS [S. Talaei and et al., 2024], FinSQL [Zhang and et al., 2024], PET-SQL [Li and et al., 2024], CHASE-SQL [M. Pourreza and et al., 2025], CRED-SQL [Shaoming Duan, 2025] y SuperSQL [B. Li and et al., 2024] adoptan estrategias análogas, recuperando muestras de valores relevantes para la pregunta. La eficacia depende de la calidad del proceso de recuperación y plantea desafíos cuando los datos contienen información sensible.

3.5.3. Conocimiento externo y de dominio

En entornos empresariales, una fracción significativa de las consultas no puede resolverse correctamente sin conocimiento externo: glosarios de métricas, definiciones de dominio, restricciones de negocio. La línea de trabajos que aborda esta necesidad es comparativamente menor pero relevante. atscale2024sml [AtScale, Inc., 2024] ilustra el uso de un *semantic layer* corporativo. SAFENLIDB [Liu et al., 2025b] aborda el componente de privacidad y seguridad de la integración con conocimiento externo. ROUTE [Y. Qin and et al., 2025] incorpora señales de robustez que implícitamente requieren conocimiento adicional sobre el dominio. La principal limitación de esta categoría es la falta de estanda-

rización: cada trabajo define su propio formato para representar el conocimiento externo, lo que dificulta la comparación.

3.5.4. Auto-refinamiento

Una proporción creciente de sistemas incorpora ciclos en los que la consulta generada se ejecuta o se inspecciona, y los resultados o los errores se reintroducen al modelo para corrección. DIN-SQL [Pourreza and Rafiei, 2023] incluye una etapa de auto-corrección. MAC-SQL [B. Wang and et al., 2023], CHESS [S. Talaei and et al., 2024], XiYan-SQL [Yingqi Gao, 2024], OpenSearch-SQL [X. Xie and Guo, 2025], DEA-SQL [Xie and et al., 2024], RSL-SQL [et al., 2024], PURPLE [T. Ren and et al., 2024] y CHASE-SQL [M. Pourreza and et al., 2025] adoptan variantes de este mecanismo. Los trabajos sobre detección de errores —ErrorLLM [Hong et al., 2026], SQLHD [Yang et al., 2025], AmbiSQL [Ding et al., 2026]— proveen señales que, en principio, podrían alimentar el bucle de auto-refinamiento. La eficacia del mecanismo depende crucialmente de la naturaleza de la retroalimentación: cuando es texto libre, tiende a inducir oscilaciones; cuando es estructurada, puede converger más rápidamente.

3.5.5. Observaciones críticas

El *schema linking* es el mecanismo más estudiado, mientras que la integración de conocimiento externo de dominio es claramente el menos desarrollado pese a su relevancia empresarial directa. El auto-refinamiento ha proliferado, pero la mayoría de las implementaciones operan sobre la consulta final como texto, lo que limita la precisión de la corrección posible y dificulta su análisis sistemático. Una arquitectura que opere sobre una RI tipada permite sustituir el bucle de auto-corrección sobre texto por retroalimentación estructurada que actúe sobre el nodo o la condición exacta donde se detectó el error.

3.6. Paradigma de Razonamiento

Esta sección agrupa los sistemas según cómo estructuran el proceso de inferencia que conduce de la pregunta a la consulta. El paradigma de razonamiento es independiente de la arquitectura del sistema: una arquitectura monolítica puede emplear razonamiento directo o en cadena, y un *pipeline* modular puede operar de forma descompositiva o iterativa.

3.6.1. Razonamiento directo

El modelo produce la consulta sin pasos intermedios explícitos. Es la estrategia natural en sistemas *end-to-end*: Seq2SQL [Victor Zhong, 2017], BRIDGE [Lin and et al., 2020], RAT-SQL [B. Wang and et al., 2021], RASAT [J. Qi and et al., 2022], PICARD [Scholak and et al., 2022], Graphix-T5 [Li and et al., 2022], RESDSQL [H. Li and Chen, 2023], T5-3B+NatSQL [Gan and et al., 2023] y CodeS [H. Li and et al., 2024]. Estos sistemas

confían el razonamiento al proceso interno del modelo y a las representaciones intermedias implícitas que aprende durante el entrenamiento.

3.6.2. Cadena de pensamiento

Aquí el modelo expone, antes de la consulta, una traza textual del razonamiento que conduce a ella. ACT-SQL [Zhang and et al., 2023] es una referencia explícita en esta línea, con *prompts* que inducen *chain-of-thought* adaptado a *text-to-SQL*. ODIS [Chang and Fosler-Lussier, 2023], DAIL-SQL [D. Gao and et al., 2023], PET-SQL [Li and et al., 2024], SuperSQL [B. Li and et al., 2024], CoE-SQL [H. Zhang and et al., 2024] y CRED-SQL [Shaoming Duan, 2025] adoptan variantes del enfoque. La principal limitación es que la traza textual no es verificable estructuralmente; el modelo puede producir un razonamiento aparentemente válido y aun así generar una consulta errónea.

3.6.3. Razonamiento descompositivo

La pregunta se divide explícitamente en sub-preguntas, cada una de las cuales se resuelve por separado y se ensambla luego. DIN-SQL [Pourreza and Rafiei, 2023] es la referencia principal en esta línea. MAC-SQL [B. Wang and et al., 2023], CHESS [S. Talaei and et al., 2024], DEA-SQL [Xie and et al., 2024], RSL-SQL [et al., 2024], TA-SQL [Qu et al., 2024], SuperSQL [B. Li and et al., 2024] y SteinerSQL [Xutao Mao, 2025] la adoptan en variantes propias. La descomposición es coherente con la observación práctica de que las consultas complejas se resuelven mejor cuando se atacan por partes, aunque introduce el riesgo de descomposiciones incorrectas que arrastran errores difíciles de localizar.

3.6.4. Razonamiento de múltiples caminos

El sistema produce varias trayectorias o varios candidatos en paralelo y elige entre ellos. CHASE-SQL [M. Pourreza and et al., 2025], XiYan-SQL [Yingqi Gao, 2024], MetaSQL [Y. Fan and et al., 2024], PARSQL [Yaxun Dai, 2025], G^3R [Y. Xiang and et al., 2023] y N-best List Rerankers [L. Zeng and Hakkani-Tur, 2022] pertenecen a esta sub-categoría. La motivación está sustentada por la observación empírica de que el candidato más probable según el modelo no es siempre el correcto; la diversidad del conjunto generado actúa como mecanismo de tolerancia a errores. La dificultad reside en construir un seleccionador o reordenador que efectivamente identifique el candidato correcto.

3.6.5. Razonamiento de retroalimentación iterativa

La consulta se refina en sucesivas iteraciones a partir de retroalimentación de ejecución, validación o un agente verificador. CHESS [S. Talaei and et al., 2024] con su Unit Tester, MAC-SQL [B. Wang and et al., 2023] con su agente de refinamiento, OpenSearch-SQL [X. Xie and Guo, 2025], DIN-SQL [Pourreza and Rafiei, 2023], RSL-SQL [et al.,

2024], DEA-SQL [Xie and et al., 2024] y CHASE-SQL [M. Pourreza and et al., 2025] operan en ciclos iterativos. ErrorLLM [Hong et al., 2026] y SQLHD [Yang et al., 2025] proveen modelos de detección de errores que pueden alimentar tales ciclos. AmbiSQL [Ding et al., 2026] aborda iterativamente la resolución de ambigüedades en interacción con el usuario. La efectividad del paradigma depende de que la retroalimentación sea informativa y discriminante; en su ausencia, el ciclo puede oscilar sin converger.

3.6.6. Observaciones críticas

La descomposición y la retroalimentación iterativa han desplazado al razonamiento directo en los sistemas más recientes basados en LLMs. Sin embargo, la cadena de pensamiento y la descomposición, en sus implementaciones predominantes, operan sobre texto libre y son por lo tanto difíciles de auditar y de verificar formalmente. La investigación de esta tesis explora una estrategia complementaria: la descomposición y la retroalimentación operan sobre una RI estructurada y tipada, lo que convierte cada paso del razonamiento en un artefacto inspectable y verificable mediante reglas explícitas.

3.7. Propiedades Evaluativas Objetivo

Esta sección caracteriza la literatura por las propiedades del sistema que cada trabajo prioriza explícitamente en su evaluación. Se distinguen seis propiedades, no excluyentes, que guían el diseño y la experimentación.

3.7.1. Corrección

La corrección, medida típicamente como exactitud de ejecución (*execution accuracy*) o coincidencia exacta (*exact match*), es la propiedad históricamente dominante. Prácticamente todos los trabajos analizados la reportan: Spider [Lei et al., 2025], BIRD [Li et al., 2023] y otros benchmarks han consolidado su uso como métrica de referencia. La crítica recurrente, presente en EvaluatingRobustness [Fürst et al., 2024] y en NLinterfaces [Quamar et al., 2022], es que la exactitud sobre benchmarks controlados sobreestima el desempeño en entornos reales.

3.7.2. Eficiencia

Una fracción menor de trabajos prioriza explícitamente la eficiencia de la consulta generada, no solo la corrección de su resultado. CodeScore [Dong et al., 2024] y atscale2024sml [AtScale, Inc., 2024] contemplan métricas asociadas. SuperSQL [B. Li and et al., 2024], PET-SQL [Li and et al., 2024] y FinSQL [Zhang and et al., 2024] reportan parcialmente costos de ejecución. La VES (*Valid Efficiency Score*) introducida en BIRD ha promovido un mayor interés en la propiedad. Su relevancia es directa en entornos

empresariales, donde dos consultas correctas pueden diferir órdenes de magnitud en costo de ejecución.

3.7.3. Robustez

Pertenecen a esta categoría los trabajos que evalúan el desempeño bajo perturbaciones del esquema, paráfrasis de la pregunta o cambios en el dominio. EvaluatingRobustness [Fürst et al., 2024], ROUTE [Y. Qin and et al., 2025], SQLHD [Yang et al., 2025], ErrorLLM [Hong et al., 2026] y AmbiSQL [Ding et al., 2026] adoptan la robustez como propiedad central. Spider 2.0 [Lei et al., 2025] desplaza la evaluación hacia escenarios empresariales donde la robustez se vuelve crítica. La consolidación de esta dimensión es todavía incipiente: no existe un protocolo único de evaluación de robustez aceptado por la comunidad.

3.7.4. Adaptabilidad

La adaptabilidad mide la capacidad del sistema para generalizar a nuevos dominios o esquemas no vistos durante el entrenamiento. ZeroNL2SQL [Z. Gu and et al., 2024], FinSQL [Zhang and et al., 2024], SchemaGraphSQL [Safdarian et al., 2025] y CRED-SQL [Shaoming Duan, 2025] priorizan explícitamente esta dimensión. La preocupación es relevante en entornos empresariales donde el esquema cambia con el tiempo y donde no es viable reentrenar el sistema con cada cambio.

3.7.5. Escalabilidad

La escalabilidad considera el comportamiento del sistema ante esquemas masivos, bases de datos con miles de tablas o cargas de trabajo sostenidas. CRUSH4SQL [et al., 2023], llmfk [Tang et al., 2026], CodeS [H. Li and et al., 2024] y CHASE-SQL [M. Pourreza and et al., 2025] la abordan parcialmente. La intersección con eficiencia es directa, pero la escalabilidad introduce además consideraciones sobre la ventana de contexto del modelo y sobre el costo amortizado del *schema linking*.

3.7.6. Interpretabilidad

La interpretabilidad se refiere a la posibilidad de inspeccionar y explicar el proceso que conduce a la consulta generada. Los trabajos que adoptan RI explícita —IRNet [Guo et al., 2019], NatSQL [Gan et al., 2021]— y los que producen trazas estructuradas —DIN-SQL [Pourreza and Rafiei, 2023], CHESS [S. Talaei and et al., 2024], MAC-SQL [B. Wang and et al., 2023]— ofrecen mayor interpretabilidad por construcción. La propiedad cobra relevancia particular en aplicaciones empresariales sujetas a auditoría y gobernanza, contexto en el que la generación opaca de un sistema *end-to-end* resulta difícilmente admisible.

3.7.7. Observaciones críticas

La literatura presenta un sesgo claro hacia la corrección como propiedad casi excluyente, en detrimento de eficiencia, robustez, adaptabilidad y, sobre todo, interpretabilidad. Spider 2.0 y BIRD han contribuido a redirigir parte de la atención hacia robustez y eficiencia, pero la interpretabilidad sigue tratada de forma marginal. Para el contexto empresarial al que se orienta esta tesis, la interpretabilidad es una propiedad de primer orden: una consulta correcta pero no inspectable es difícil de operar de forma confiable. Esta consideración refuerza la decisión arquitectónica de adoptar una RI explícita y un bucle de verificación auditable.

3.8. Síntesis y Brecha de Investigación

En las secciones anteriores se ha analizado la literatura reciente bajo una taxonomía multidimensional que cubre el paradigma de aprendizaje, la arquitectura del sistema, las estrategias de representación, los mecanismos de enriquecimiento contextual, el paradigma de razonamiento y las propiedades evaluativas objetivo. La revisión permite extraer cinco observaciones de síntesis.

1. **Migración del aprendizaje al diseño arquitectónico.** Los trabajos más recientes han desplazado el énfasis del *fine-tuning* de extremo a extremo hacia configuraciones *in-context* e híbridas, complementadas con arquitecturas modulares y multiagente. La calidad final del sistema depende cada vez menos del ajuste del modelo base y cada vez más de la orquestación de componentes.

2. **Predominio de la decodificación libre sobre la representación intermedia.** La mayoría de los sistemas basados en LLMs prescinde de RI explícita y delega la corrección estructural al generador. Solo una minoría —IRNet [Guo et al., 2019], NatSQL [Gan et al., 2021], SmBoP [Rubin and Berant, 2021], PICARD [Scholak and et al., 2022], SQLFormer [A. Bazaga and et al., 2023], SteinerSQL [Xutao Mao, 2025]— adopta variantes de RI o decodificación restringida. Esta minoritariedad contrasta con la creciente exigencia de verificabilidad en entornos empresariales.

3. **Auto-refinamiento sobre texto libre.** La mayoría de los bucles de refinamiento opera sobre la consulta como texto y sobre retroalimentación textual del error de ejecución. Esta granularidad limita la precisión de la corrección y favorece oscilaciones. La retroalimentación estructurada sobre una RI tipada permanece como una vía explorada solo parcialmente.

4. **Schema linking saturado, conocimiento externo incipiente.** El *schema linking* ha recibido atención sostenida y ha alcanzado un grado de madurez considerable. En contraste, la integración con conocimiento externo de dominio —glosarios, métricas, restricciones de negocio— sigue siendo una capacidad incipiente, sin formato estandarizado y sin integración nativa con la mayoría de las arquitecturas.

5. **Cobertura nula de consultas híbridas SQL/PGQ.** Pese a la formalización del estándar ISO/IEC 9075-16:2023 [ISO/IEC JTC 1/SC 32, 2023] y a la disponibilidad de motores

que lo implementan [Wolde et al., 2023], ninguno de los sistemas analizados aborda la generación de consultas híbridas bajo este estándar.[1] STRuCT-LLM [Stoisser et al., 2025] y text2gqlbench [Lyu et al., 2026] se aproximan al razonamiento sobre grafos, pero no contemplan la dualidad relacional/grafo propia de SQL/PGQ. Otros trabajos como SchemaGraphSQL [Safdarian et al., 2025] se sitúan en la frontera, pero sin abordar la generación bajo el estándar híbrido.

De la conjunción de estas cinco observaciones se desprende una brecha clara. La literatura ha desarrollado de forma extensiva técnicas parciales para mitigar alucinaciones en *text-to-SQL*: *schema linking*, decodificación restringida, descomposición, auto-refinamiento y verificación por ejecución. Sin embargo, no existe en la literatura analizada una arquitectura que integre simultáneamente: (i) una RI explícita y tipada que cubra tanto el componente relacional como el de patrón de grafo del estándar híbrido SQL/PGQ; (ii) una decodificación restringida o composicional sobre dicha RI; (iii) un bucle de verificación que opere a nivel de RI con retroalimentación estructurada en lugar de texto libre; y (iv) propiedades evaluativas que prioricen la interpretabilidad y la robustez junto con la corrección. El siguiente capítulo presenta la propuesta de la tesis, cuyo objetivo es articular estos cuatro componentes en una arquitectura coherente que aborde explícitamente la generación confiable de consultas SQL/PGQ a partir de lenguaje natural.

[1] El alcance de la revisión cubre publicaciones indexadas en arXiv, ACL Anthology, IEEE Xplore y proceedings de ICLR, NeurIPS, EMNLP, ACL, VLDB y SIGMOD entre 2017 y 2026, con búsquedas combinando los términos *text-to-SQL*, *text-to-query*, *SQL/PGQ*, *property graph*, *hybrid graph relational query* y sus variantes.

Capítulo 4

Propuesta: Arquitectura basada en Representación Intermedia para Generación Confiable de Consultas SQL/PGQ

A través del desarrollo de los anteriores capítulos, podemos evidenciar una brecha de investigación claramente formulada: Las propuestas en el campo de text-to-query disponen de piezas maduras para abordar facetas individuales del problema *text-to-query*, como pueden ser: la descomposición del proceso de generación en etapas, la decodificación sujeta a restricciones, la verificación por ejecución y el enlazamiento con el esquema.

No obstante, tales mecanismos han sido estudiados principalmente de manera fragmentada y sin tomar en consideración la aplicación extendida a consultas híbridas que combinen el paradigma relacional con relaciones de grafo conforme al estándar SQL/PGQ [ISO/IEC JTC 1/SC 32, 2023], el cual es uno de los estandáres con mayor interés creciente actualmente en la industria. A partir de esta observación, el presente capítulo desarrolla la propuesta de investigación orientada a cerrar dicha brecha mediante una integración arquitectónica de estos aportes en el contexto de la generación de consultas empresariales sobre esquemas híbridos.

La propuesta se presenta como un trabajo experimental e incremental, siendo que se articula una arquitectura cuya implementación se construye por fases debidamente delimitadas sobre componentes preexistentes revisados en al literatura.

4.1. Bases técnicas y criterios de selección

Para desarrollar la propuesta se tomaron como base seis elementos del estado del arte y del ecosistema de bases de datos: tres provenientes de la literatura *text-to-SQL* (XiYan-SQL, CHESS, RSL-SQL), uno del ecosistema relacional con extensión a grafos (DuckDB con la extensión DuckPGQ), uno como marco de evaluación canónico (NL2SQL360) y uno

como infraestructura de análisis estático de SQL (sqlglot). Estos elementos no se eligieron por exhaustividad sino por satisfacer simultáneamente cinco criterios, formulados aquí de manera explícita para que la decisión sea reproducible:

1. **Cobertura conceptual de al menos una dimensión del marco taxonómico.** Cada base aborda con detalle al menos una de las seis dimensiones definidas en la Sección 3.1 (paradigma de aprendizaje, arquitectura, representación, enriquecimiento, razonamiento o evaluación), de modo que la integración de las bases cubre el espectro relevante para la propuesta.
2. **Compatibilidad con el estándar SQL/PGQ.** La base debe poder integrarse en un *pipeline* cuya salida final es una consulta sobre el estándar ISO/IEC 9075-16:2023 [ISO/IEC JTC 1/SC 32, 2023], ya sea como motor de ejecución directo o como componente cuya salida pueda traducirse a ese dialecto.
3. **Modificabilidad por puntos de extensión.** Cada base admite extensión sin modificación intrusiva del código fuente, mediante adaptadores, herencia o configuración. Las modificaciones requeridas se documentan en la Sección 4.5.
4. **Reproducibilidad operacional.** Cada base puede instalarse con dependencias resueltas de forma determinista. Las versiones de las dependencias críticas se documentan en la Sección 4.5.
5. **Accesibilidad y licencia compatible.** El componente está disponible bajo una licencia que permite su integración en un prototipo académico revisable, con código fuente público y documentación suficiente para reproducir su uso. Este criterio excluye sistemas comerciales sin acceso al código, los cuales se discuten cualitativamente, pero no se reutilizan como base experimental.

A continuación se describe cada base, con la razón específica que motiva su elección además de los criterios comunes anteriores.

XiYan-SQL [Yingqi Gao, 2024]. Sistema multi-generador de *text-to-SQL*, del cual se reutiliza, en primera instancia, la representación **M-Schema**: un formato semi-estructurado del esquema relacional que combina nombres, tipos, claves primarias, claves foráneas y muestras de valores en un bloque presentable directamente al LLM. M-Schema es una de las representaciones de esquema más adoptadas en la literatura reciente, y constituye el punto de partida natural para extender el formato a esquemas duales. El propio repositorio público de XiYan-SQL es un meta-repositorio que enlaza a los componentes ejecutables (modelos `XiYanSQL-QwenCoder`, marco de entrenamiento, servidores MCP); este hecho, descubierto durante la fase de integración, condicionó la decisión de incorporar XiYan-SQL primero como referencia de diseño y posponer la integración de los componentes ejecutables.

CHESS [S. Talaei and et al., 2024]. Sistema modular con razonamiento contextual y verificación basada en ejecución. Se utiliza como *baseline* arquitectónico comparativo y como fuente conceptual de dos mecanismos: (a) la recuperación de valores de la base de datos mediante búsqueda por similitud aproximada, que se adapta para recuperar también

valores asociados a propiedades de vértices y aristas del grafo de propiedad; y (b) un esquema de verificación dinámica con retroalimentación al generador, que se reformula en esta tesis bajo el principio de mediación obligatoria por representación intermedia (P2, Sección 4.2).

RSL-SQL [et al., 2024]. Trabajo orientado a *schema linking* robusto sobre esquemas de gran escala. Aporta la idea del enlazado bidireccional —una pasada hacia adelante (pregunta → esquema) y otra hacia atrás (esquema → menciones)—, que se adopta y se generaliza al esquema dual (relacional + grafo de propiedad). RSL-SQL no se reutiliza como ejecutable: se reutiliza como descripción de un mecanismo, que se reimplementa de forma independiente para integrar las anotaciones de grafo.

DuckDB con la extensión DuckPGQ [Wolde et al., 2023]. DuckDB es un motor relacional empotrado de propósito analítico; la extensión DuckPGQ implementa la sub-cláusula `GRAPH_TABLE(... MATCH ... COLUMNS (...))` del estándar SQL/PGQ compilándola internamente a operadores relacionales. Esta combinación es, en el momento de redacción, la única implementación abierta que ejecuta SQL/PGQ estandarizado de forma directa, con persistencia nativa de los grafos de propiedad. Se elige por satisfacer la condición de "ejecutabilidad validable sobre el estándar", sin la cual la verificación dinámica operaría sobre una aproximación.

NL2SQL360 [B. Li and et al., 2024]. Marco de evaluación revisado por pares para sistemas *text-to-SQL* sobre los *benchmarks* Spider y BIRD. Se incorpora sin modificaciones para producir las métricas canónicas (*Execution Accuracy*, *Exact Match*, *Valid Efficiency Score*) en el régimen relacional puro.

sqlglot. Biblioteca de *parsing* y transformación de SQL en Python (Tobias Mao, `github.com/tobymao/sqlglot`) que soporta múltiples dialectos, incluido `duckdb`. Se utiliza como *front-end* sintáctico del verificador estático: produce un árbol de sintaxis abstracta sobre el cual se aplican las reglas locales de existencia de tablas, columnas y compatibilidad de aliases. La elección obedece a la madurez de la biblioteca y a su soporte explícito de DuckDB; no se reutiliza ningún parser *ad hoc*.

La Figura 4.1 resume las bases seleccionadas y la dimensión del marco taxonómico que cubre cada una. Las secciones siguientes hacen referencia a estos elementos por nombre, con el fin de mantener visible la procedencia.

4.2. Requisitos y principios de diseño

Los requisitos que se formulan en esta sección no constituyen una enumeración arbitraria, sino la traducción operacional de tres insumos previos del documento: (i) el planteamiento del problema desarrollado en el Capítulo 1, (ii) los fundamentos conceptuales

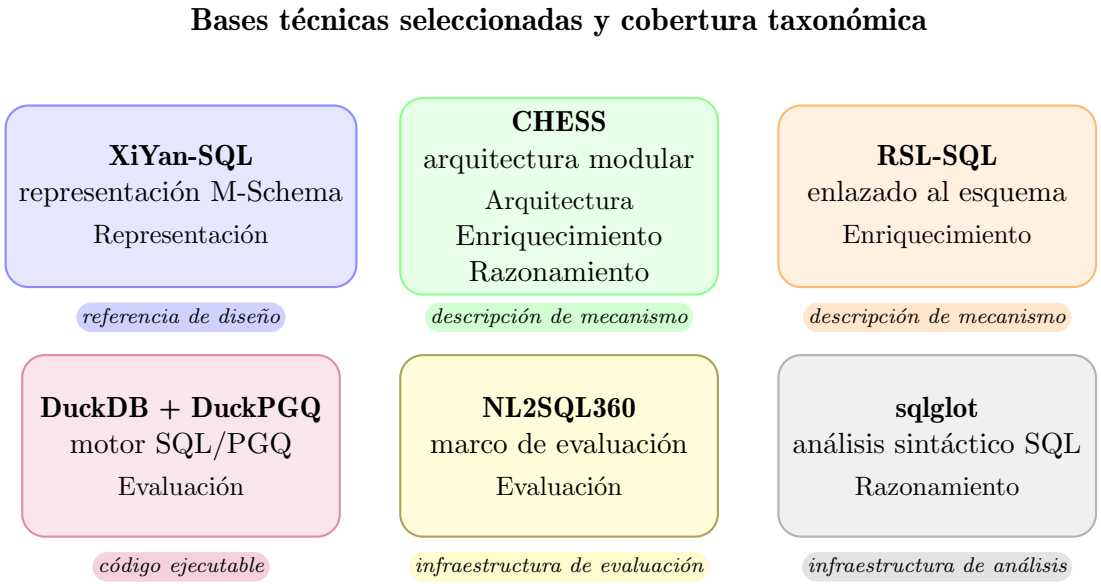

Figura 4.1: Bases técnicas reutilizadas para construir la propuesta. Cada base satisface los criterios comunes de selección y aporta a una dimensión específica del marco taxonómico definido en la Sección 3.1.

revisados en el Capítulo 2, y (iii) las brechas identificadas en la síntesis del estado del arte. Cada requisito se identifica mediante un código (R_i) y se asocia con la dimensión del problema a la que responde.

R1. Anclaje estructural al esquema dual. La arquitectura debe producir consultas cuya estructura pueda verificarse contra un esquema dual compuesto por: (a) el esquema relacional, que incluye tablas, columnas, claves foráneas y tipos; y (b) el grafo de propiedad superpuesto, que comprende la definición de vértices, aristas, etiquetas y propiedades clave–valor conforme a la declaración `CREATE PROPERTY GRAPH` del estándar [ISO/IEC JTC 1/SC 32, 2023, Wolde et al., 2023]. Dicha verificabilidad debe ser posible antes de la ejecución de la consulta, y no solo como consecuencia de un error retornado por el motor.

R2. Representación intermedia tipada. La generación no debe producir SQL/PGQ de manera directa a partir del lenguaje natural. Debe mediar una RI explícita y tipada, cuya correctitud estructural sea decidible mediante reglas locales y cuya compilación a SQL/PGQ sea determinista. Este requisito traslada al contexto dual la lección central de IRNet [Guo et al., 2019] y RESDSQL [H. Li and Chen, 2023].

R3. Separación entre generación estadística y compilación determinista. La arquitectura debe preservar una separación estricta entre: (a) el componente estadístico encargado de producir hipótesis de interpretación a partir del lenguaje natural; y (b) el componente determinista responsable de compilar dichas hipótesis a SQL/PGQ. El propósito de esta separación es confinar las posibles alucinaciones del modelo al espacio de la RI, de modo que puedan detectarse y rechazarse antes de alcanzar el motor de ejecución.

R4. Verificación en dos regímenes. La propuesta debe incorporar un bucle de verificación que combine: (a) verificación estática sobre la RI y sobre la SQL/PGQ compilada, incluyendo referencias a elementos existentes, consistencia de tipos y coherencia entre los componentes relacionales y de grafo; y (b) verificación dinámica mediante ejecución controlada en un entorno de validación con DuckPGQ como motor de referencia [Wolde et al., 2023]. Este requisito retoma las propuestas de verificación revisadas en la Sección 3.4.3 y amplía su alcance al dominio híbrido.

R5. Modularidad y agnosticismo respecto del LLM. Los componentes de generación deben acoplarse al LLM mediante una interfaz abstracta que permita sustituir el modelo subyacente sin modificar la arquitectura general. Este requisito responde a la observación empírica de que los resultados reportados en el estado del arte dependen de manera crítica del LLM utilizado, fenómeno documentado por SuperSQL [B. Li and et al., 2024] y por la proliferación de enfoques que combinan múltiples generadores [M. Pourreza and et al., 2025, Yingqi Gao, 2024].

R6. Trazabilidad de decisiones intermedias. Cada consulta generada debe poder rastrearse a lo largo de sus etapas intermedias: interpretación del lenguaje natural, construcción de la RI, compilación y verificación. La trazabilidad constituye un requisito previo para la interpretabilidad en entornos empresariales, una propiedad que el estado del arte actual aborda solo de manera marginal [Adedeji and Mabere, 2023, Quamar et al., 2022].

R7. Portabilidad y agnosticismo del motor de ejecución. La arquitectura debe ser transferible a esquemas y motores distintos del entorno de validación inicial sin reescritura del *pipeline*. Las restricciones específicas del motor de referencia —por ejemplo, las inducidas por la implementación actual de DuckPGQ [Wolde et al., 2023]— deben quedar aisladas en componentes de configuración del compilador y del verificador, no en la RI ni en la lógica de orquestación. Este requisito responde a la heterogeneidad de los entornos empresariales, donde coexisten distintos RDBMSs y donde la viabilidad práctica de la propuesta depende de su capacidad para absorber esa heterogeneidad sin rediseño.

R8. Coste de inferencia acotado. El procesamiento de una consulta debe operar bajo un presupuesto explícito de invocaciones al LLM, tokens consumidos y latencia agregada. El bucle de refinamiento debe incorporar un criterio de terminación que limite el número máximo de iteraciones, de modo que el coste agregado por consulta sea predecible y comparable con el de los sistemas del estado del arte. Este requisito materializa la observación de que la adopción empresarial de interfaces en lenguaje natural depende tanto de la corrección como del coste por consulta sostenido a escala.

A partir de estos ocho requisitos se derivan cinco *principios de diseño* que orientan las decisiones arquitectónicas concretas de la propuesta.

P1. Grounding antes que generación. Toda generación debe estar precedida por una fase explícita de anclaje al esquema dual. El modelo no genera SQL/PGQ sobre un esquema opaco, sino sobre una representación previamente proyectada y validada.

P2. Mediación obligatoria por representación intermedia. Ningún artefacto sintáctico de consulta alcanza el motor de ejecución sin pasar por la RI tipada. La RI constituye la frontera formal entre el componente estadístico y el componente determinista; la traducción desde la RI hacia SQL/PGQ es una función determinista, sin intervención adicional del modelo.

P3. Verificación en cascada. La validación de las consultas se organiza en niveles de costo creciente: la verificación estática precede a la dinámica, y solo las consultas que superan el nivel anterior son sometidas al siguiente. Esta progresión reduce el costo computacional y minimiza efectos colaterales sobre el RDBMS.

P4. Transparencia estructural. Cada componente debe exponer su entrada, su salida y su criterio de decisión en un formato auditable. De este modo, la opacidad propia del *prompting* monolítico se sustituye por una traza estructurada e inspeccionable.

P5. Desacoplamiento del modelo generador. El LLM se trata como una dependencia sustituible: los componentes de generación lo invocan a través de una interfaz abstracta, y ninguna decisión arquitectónica queda atada a un modelo específico.

Estos principios se concretan en el *pipeline* descrito a continuación.

4.3. Flujo metodológico general

El *pipeline* propuesto se articula en cinco etapas secuenciales, junto con un bucle de refinamiento asimétrico entre la construcción de la RI y la fase de verificación. La Figura 4.2 resume el flujo metodológico general de la propuesta.

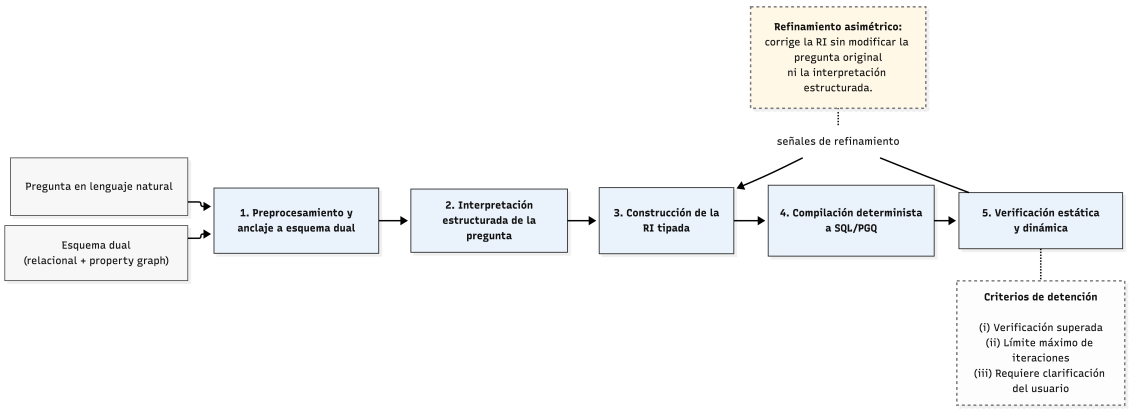

Figura 4.2: Flujo metodológico general del *pipeline* propuesto. Las cinco etapas secuenciales procesan la pregunta en lenguaje natural y el esquema dual (relacional y grafo de propiedad) hasta producir una consulta SQL/PGQ verificada.

1. **Pre-procesamiento y anclaje al esquema dual.** Esta etapa recibe como entrada la pregunta en lenguaje natural, el esquema relacional y la definición del grafo de propiedad. Su salida es un esquema proyectado, es decir, un subconjunto del esquema

dual enriquecido con señales de relevancia, tales como tablas y columnas candidatas, vértices y aristas potencialmente implicados, así como *cell-values* relevantes. Esta fase materializa el principio P1 y se apoya en ideas de anclaje al esquema y reducción del espacio de búsqueda reportadas en [et al., 2024, S. Talaei and et al., 2024].

2. **Interpretación estructurada de la pregunta.** A partir de la pregunta y del esquema proyectado, esta etapa produce una descomposición explícita de la intención del usuario en componentes interpretables: entidades mencionadas, restricciones, proyecciones esperadas y tipo de razonamiento requerido. En particular, identifica si la consulta demanda razonamiento relacional, razonamiento sobre grafos o una combinación de ambos. Esta etapa se inspira en el paradigma de descomposición presentado en [Pourreza and Rafiei, 2023, B. Wang and et al., 2023], extendiéndolo al contexto híbrido SQL/PGQ.

3. **Construcción de la representación intermedia tipada.** Con base en la interpretación estructurada, se construye una instancia de la RI propuesta. Esta etapa constituye el núcleo conceptual de la arquitectura, pues traduce la intención del usuario a una forma estructurada, tipada y validable antes de cualquier compilación al lenguaje de consulta final.

4. **Compilación determinista a SQL/PGQ.** Una vez que la RI satisface sus restricciones estructurales, se compila a una consulta concreta mediante reglas deterministas, sin intervención adicional del LLM. De este modo, la generación estadística queda separada de la síntesis sintáctica final, y la consulta producida se ajusta por construcción a la gramática del estándar objetivo.

5. **Verificación estática y dinámica.** La consulta compilada se somete, en primer lugar, a una verificación estática orientada a comprobar su consistencia estructural respecto del esquema dual. Si supera esta fase, se procede a una verificación dinámica mediante ejecución controlada sobre DuckPGQ [Wolde et al., 2023]. Los errores detectados en cualquiera de estos dos niveles se transforman en señales de refinamiento que retroalimentan la etapa de construcción de la RI.

El bucle de refinamiento es asimétrico en el sentido que la retroalimentación no modifica la pregunta original ni reabre la interpretación de alto nivel de la etapa dos, sino que actúa exclusivamente sobre la construcción de la RI. Esta decisión responde al principio P2, ya que concentra la corrección en el espacio donde la estructura puede manipularse de forma explícita y verificable mediante reglas. El refinamiento concluye cuando se satisface alguno de los siguientes criterios: (i) la verificación es superada, (ii) se alcanza un número máximo de iteraciones, o (iii) el sistema determina que la pregunta presenta una ambigüedad intrínseca que requiere clarificación del usuario, en consonancia con lo planteado por [Ding et al., 2026].

4.4. Arquitectura y componentes

Esta sección detalla los cuatro componentes arquitectónicos que materializan el *pipeline*: el anclaje al esquema dual (§4.4.1), la RI tipada IR-SQL/PGQ (§4.4.2), el compilador determinista (§4.4.3) y el bucle de verificación (§4.4.4).

4.4.1. Anclaje al esquema dual

El primer componente resuelve un problema estructural que los sistemas *text-to-SQL* clásicos no enfrentan: la pregunta en lenguaje natural debe proyectarse sobre dos estructuras superpuestas. Un mismo concepto lingüístico puede corresponder simultáneamente a una tabla relacional y a una etiqueta de vértice en el grafo de propiedad. La arquitectura necesita identificar cuál de las dos proyecciones es relevante, o si ambas lo son, antes de proceder a la generación.

Para ello, se define un *esquema proyectado*

$$\Sigma_{\text{proj}} \;=\; \langle T_{\text{rel}},\, V_{\text{grafo}},\, E_{\text{grafo}},\, C_{\text{val}} \rangle,$$

donde T_{rel} es el subconjunto relevante de tablas y columnas relacionales, V_{grafo} y E_{grafo} son los subconjuntos de tipos de vértice y arista del grafo de propiedad, y C_{val} es el conjunto de valores recuperados de la base que coinciden con menciones literales en la pregunta. La construcción de Σ_{proj} se apoya en tres mecanismos complementarios:

- **Representación enriquecida del esquema.** Se adopta y extiende el formato M-Schema introducido por XiYan-SQL [Yingqi Gao, 2024], agregando descriptores de grafo (declaraciones `VERTEX TABLES` y `EDGE TABLES`) al bloque semi-estructurado que se presenta al LLM. La extensión se realiza fuera del repositorio original, en el módulo propio `core/anchor/m_schema_dual.py`.
- **Enlazado bidireccional.** Inspirado en RSL-SQL [et al., 2024], el enlazado se ejecuta en dos pasadas y retiene únicamente los elementos del esquema corroborados en ambas direcciones. La implementación es propia; RSL-SQL se reutiliza como descripción de mecanismo, no como código.
- **Recuperación de valores.** Se adopta la idea de búsqueda por similitud aproximada propuesta en CHESS [S. Talaei and et al., 2024] y se adapta para recuperar también valores asociados a propiedades de vértices y aristas del grafo de propiedad.

El esquema proyectado es el único contexto estructural que las etapas posteriores consumen sobre la base de datos: la arquitectura no admite que el LLM invoque elementos ausentes de Σ_{proj} al construir la RI.

4.4.2. Representación intermedia tipada (IR-SQL/PGQ)

La RI propuesta, denominada **IR-SQL/PGQ**, es la pieza central de la arquitectura. Su valor es triple. Constituye el contrato fijo entre el modelo de lenguaje y el resto del *pipeline*, ya que el modelo no produce SQL/PGQ libre sino una instancia de RI cuya estructura está validada al momento de la generación. Es la única estructura sobre la cual el verificador estático opera, sin necesidad de parsear SQL ni de inferir desde un árbol de sintaxis abstracta del motor. Es la entrada del compilador determinista que materializa la consulta concreta, con la garantía formal de que dos instancias estructuralmente idénticas de la RI producen el mismo SQL textual.

Posicionamiento respecto a representaciones intermedias previas. IR-SQL/PGQ se sitúa en el linaje de las representaciones intermedias tipadas para *text-to-SQL* y hereda principios de tres familias del estado del arte. De SemQL [Guo et al., 2019] hereda la idea de descomponer la generación en una secuencia de decisiones tipadas sobre un esqueleto, donde cada acción expande un componente con un tipo predefinido. De NatSQL [Gan et al., 2021] hereda el principio de colapsar varios constructos del lenguaje en operaciones equivalentes pero más uniformes, lo que reduce la superficie de decisión del modelo. De los *skeletons* de RESDSQL [H. Li and Chen, 2023] hereda la disciplina operativa de mantener un compilador determinista descendiente de la RI a la consulta textual. La novedad declarada de IR-SQL/PGQ frente a estas tres familias es doble: el tratamiento uniforme de bloques de grafo (`MatchPattern`, `PathPattern`, `VertexPattern`, `EdgePattern`) que permite expresar consultas SQL/PGQ con la misma economía sintáctica con la que SemQL expresa consultas relacionales; y la composicionalidad híbrida (`FromGraphMatch`), que permite que el resultado de una consulta de grafo participe como tabla derivada en el contexto relacional sin requerir nodos especiales para el caso mixto.

Estructura general. La RI es un árbol cuyos nodos están representados como *dataclasses* inmutables. La inmutabilidad es deliberada y garantiza tres propiedades operativas: las instancias son *hashables* y se pueden usar como claves de estructuras auxiliares; la igualdad es estructural sin necesidad de redefinir el operador de comparación; y no hay riesgo de *aliasing* accidental, ya que cualquier modificación exige construir una nueva instancia. Las colecciones internas se representan como tuplas en lugar de listas, lo que se traduce directamente a *arrays* en el formato JSON de serialización descrito más adelante. Ningún nodo contiene lógica de método: son contenedores puros de datos sobre los cuales operan, externamente, el lifter, el compilador y el verificador.

La RI se organiza en tres bloques: el bloque relacional, el bloque de grafo y la composición híbrida. La unión es cerrada en el sentido de que toda consulta dentro del alcance de la propuesta es expresable como composición de estos tres bloques.

Bloque relacional. Cubre el subconjunto de SQL ANSI que aparece en los *benchmarks* de evaluación relevantes (Spider, BIRD). Se estructura en cinco capas: referencias a esquema (`TableRef`, `ColumnRef`); expresiones, que incluyen literales tipados, columnas, comodines, operadores binarios y unarios, llamadas a funciones escalares y de agregación con marcador opcional de `DISTINCT`, expresiones condicionales, conversiones de tipo, patrones de comparación de cadenas, pruebas de pertenencia y nulidad, rangos, existenciales sobre subconsultas, subconsultas como expresión y agrupación explícita por paréntesis; ítems de cláusula (`SelectItem`, `OrderItem`); fuentes del FROM (`FromTable`, `FromSubquery`, `Join`); y consultas de nivel superior (`RelationalQuery`, `SetOperation`, esta última cubre `UNION`, `INTERSECT` y `EXCEPT` con sus variantes `ALL`). Las expresiones son recursivas y admiten anidamiento arbitrario, lo que permite representar combinaciones que el lenguaje natural genera sobre dominios reales.

La cobertura empírica del bloque relacional, medida sobre las cien consultas predichas por el modelo de lenguaje en el experimento de *baseline* sobre Spider *dev*, alcanza el noventa y nueve por ciento de *round-trip* estructural exacto; el caso restante corresponde a una *window function*, construcción que la RI todavía no representa y que está documentada

como extensión natural pendiente.

Bloque de grafo. Materializa la cláusula `GRAPH_TABLE(... MATCH ... COLUMNS (...))` del estándar SQL/PGQ [ISO/IEC JTC 1/SC 32, 2023, Wolde et al., 2023]. Consta de cuatro nodos. `VertexPattern` representa un patrón de vértice con su variable de enlace y su etiqueta de tipo. `EdgePattern` representa un patrón de arista con su variable, etiqueta y dirección, esta última admitiendo tres valores literales (forward, backward o no dirigida) que invierten o relajan los roles de *source* y *destination* declarados por la arista en el catálogo del grafo. `PathPattern` agrupa un vértice cabecera y una secuencia ordenada de pasos arista-vértice, lo que permite expresar caminos *multihop*. `MatchPattern` es la raíz del bloque y agrupa un identificador del *property graph* consultado, una colección de patrones de camino, una cláusula `WHERE` interna sobre las variables del MATCH, y una colección de `SelectItem` en la cláusula `COLUMNS` que define las columnas que el bloque expone hacia afuera. La superficie expuesta por `COLUMNS` es la única vía por la que las variables del MATCH se proyectan al contexto exterior; el resto permanece encapsulado.

Composición híbrida. El nodo `FromGraphMatch` materializa la unión entre los bloques relacional y de grafo. Como elemento de la unión `FromExpr`, puede ocupar cualquier posición que un `FromTable` ocuparía: ser la única fuente de un `RelationalQuery` (caso de consulta puramente de grafo), ser un operando de un `Join` con una tabla relacional (caso híbrido canónico, donde el resultado del MATCH se mezcla con datos relacionales mediante `JOIN ... ON`), o ser un operando de otro `Join` anidado. La composicionalidad es estructural: no hay nodos especiales para el caso híbrido, y todas las operaciones del bloque relacional (filtros, agrupaciones, ordenamientos, agregaciones, *set operations*) siguen siendo expresables uniformemente sobre el resultado del MATCH.

Esta uniformidad es el punto donde la RI difiere de las representaciones previas, todas concebidas para SQL relacional puro: los esquemas como SemQL y NatSQL no contemplan bloques de grafo y los *skeletons* de RESDSQL no admiten la heterogeneidad sintáctica entre las cláusulas que la composición híbrida exige.

El alias del `FromGraphMatch` cumple un rol específico en la verificación. Las variables internas del MATCH no se propagan al *scope* del `RelationalQuery` contenedor; lo que sí se propaga son las columnas declaradas en `MatchPattern.columns`, accesibles desde el contexto exterior mediante la sintaxis `<alias>.<columna>`. El verificador estático mantiene tres ámbitos separados (aliases de tablas relacionales, aliases de bloques de grafo, variables de vértice del MATCH) para resolver correctamente cada calificador a su origen. Esta separación de ámbitos, no presente en la verificación de *text-to-SQL* clásica, es necesaria para evitar tanto falsos positivos al rechazar referencias válidas como falsos negativos al admitir referencias que escapan del MATCH.

Esquema proyectado y catálogo de grafo. El verificador y el compilador consumen el esquema proyectado Σ_{proj} definido en la Sección 4.4.1 junto con un catálogo del grafo de propiedad que declara, para cada *label* de vértice, la tabla relacional asociada y las columnas clave, y para cada *label* de arista, las tablas y los *labels* de *source* y *destination* con sus respectivas claves. Esta declaración corresponde directamente a la

cláusula `CREATE PROPERTY GRAPH ... VERTEX TABLES (...) EDGE TABLES (...)` del estándar [ISO/IEC JTC 1/SC 32, 2023, Wolde et al., 2023]. La unión del esquema relacional, el catálogo de grafos de propiedad y los valores recuperados constituye el único contexto estructural que el verificador consume; la RI se valida exclusivamente contra ese contexto, sin que el verificador inspeccione la base de datos directamente.

Contrato JSON con el modelo de lenguaje. La RI tiene un contrato bidireccional con un formato JSON plano que materializa el principio P2 (mediación obligatoria). Cada nodo se representa como un objeto JSON con un campo `type` cuyo valor es el nombre exacto de la *dataclass*; las uniones discriminadas (`Query`, `Expression`, `FromExpr`) resuelven por ese campo. El JSON Schema correspondiente se construye en formato *draft 2020-12*[1] y se utiliza como `input_schema` de una *tool* en la Application Programming Interface (API) del modelo de lenguaje, lo que fuerza una salida estructurada y delega la primera validación a la **SDK!** (**SDK!**) antes de que el *payload* alcance el *pipeline*.

El contrato JSON es lo que materializa la propiedad de cierre estructural promovida por el principio P2: el modelo no puede emitir SQL/PGQ libre porque la única salida válida del *tool* es un objeto RI; el *schema* valida la forma; el *parser* valida la coherencia con las *dataclasses*; el verificador estático valida la coherencia con el esquema proyectado. Cuatro guardas en serie reducen el universo de fallos posibles a errores semánticos no detectables por análisis estático, que quedan para el régimen de verificación dinámica (Sección 4.4.4). Este encadenamiento de guardas es lo que distingue la propuesta de los enfoques de auto-corrección puros del estado del arte, donde el modelo emite SQL textual y la corrección se aplica *ex post* sobre la salida; aquí, en cambio, la corrección se aplica *ex ante* sobre la RI, antes de que cualquier SQL textual exista.

Invariantes operativos. Cuatro invariantes sostienen el comportamiento del sistema y son los que las pruebas y los experimentos validan empíricamente.

La *inmutabilidad e igualdad estructural* se desprende directamente de la elección de tipo (*dataclasses* `frozen` con campos primitivos o tuplas) y es la propiedad que garantiza el determinismo del compilador.

El *round-trip de lift y compilación* afirma que para todo `Query` producido por el lifter sobre un SQL textual, la RI resultante de re-liftar el SQL compilado es estructuralmente idéntica al original módulo nodos cosméticos de paréntesis; es la versión empírica de la afirmación de que el compilador es la inversa derecha del lifter, y se valida sobre las cien consultas del experimento de *baseline*.

El *round-trip de parse y serialización* afirma que la composición `parse_ir(to_dict(q)) == q` preserva la igualdad estructural; es la propiedad que cualquier RI producida por el modelo y validada por el *schema* debe satisfacer.

La cuarta invariante es la *soundness referencial*: si una RI pasa el verificador estático contra Σ_{proj}, entonces toda referencia a tabla, columna, *label* de vértice, *label* de arista

[1] JSON Schema, especificación draft 2020-12, `https://json-schema.org/draft/2020-12`. Cita canónica pendiente de incorporar al BibTeX.

o propiedad en la RI corresponde a un elemento declarado en Σ_{proj}. Es la formulación operativa de la propiedad declarada en el principio P2; la condición inversa (*completeness*) no se garantiza, ya que el verificador estático puede dejar pasar consultas que el motor luego rechaza por errores semánticos no previstos, lo que motiva la fase dinámica del bucle de verificación.

4.4.3. Compilador determinista a SQL/PGQ

El compilador transforma una instancia válida de IR-SQL/PGQ en una consulta concreta en dialecto SQL/PGQ. La operación es determinista en sentido estricto: dos instancias estructuralmente idénticas de la RI producen la misma consulta compilada, independientemente de estados externos o de llamadas al LLM.

La compilación procede por recorrido ascendente del árbol. Los bloques relacionales se traducen a SQL estándar siguiendo reglas análogas a las de NatSQL [Gan et al., 2021]. Los bloques de grafo se traducen a la cláusula `FROM GRAPH_TABLE(... MATCH ... COLUMNS (...))` del estándar [Wolde et al., 2023]; los bloques híbridos se materializan como subconsultas donde el resultado de `GRAPH_TABLE` participa como tabla derivada en el contexto relacional. Una restricción operativa heredada del motor merece mención: en la versión actual de DuckPGQ, los patrones `MATCH` exigen una variable explícita en cada arista (por ejemplo, `(a:Person)-[k:knows]->(b:Person)` en lugar de `(a:Person)-[:knows]->(b:Person)`); el compilador genera dicha variable de forma sistemática aunque no se utilice posteriormente, decisión documentada en el registro técnico del prototipo.

El compilador no realiza optimización. La consulta producida puede no ser la más eficiente posible, pero es válida por construcción respecto al estándar y al esquema. La optimización se delega al motor de ejecución (DuckDB con la extensión DuckPGQ), siguiendo el principio de separación de responsabilidades.

4.4.4. Bucle de verificación

La verificación se organiza en dos regímenes encadenados, conforme al principio P3.

Verificación estática. Opera sobre IR-SQL/PGQ sin ejecutarla y comprende cinco clases de chequeo organizadas en cuatro familias. Las primeras tres familias son contribución del enfoque y aplicarían a cualquier motor que implemente el estándar SQL/PGQ; la cuarta familia agrupa restricciones operativas específicas del motor objetivo y se mantiene separada conceptualmente para preservar la coherencia del discurso entre diseño abstracto e ingeniería de portabilidad.

Familia referencial. Toda referencia a una tabla, columna, calificador, grafo, etiqueta de vértice o etiqueta de arista debe corresponder a un elemento declarado en Σ_{proj}. Una referencia no resuelta indica una alucinación referencial del modelo. Los descriptores canónicos emitidos son `unknown_table`, `unknown_column`, `unknown_qualifier`,

`unknown_graph`, `unknown_vertex_label` y `unknown_edge_label`. La resolución es *case-insensitive*, congruente con el comportamiento por defecto de los motores objetivo sobre nombres de tabla y columna.

Familia de consistencia de tipos. Los operadores se aplican únicamente sobre operandos cuyos tipos los admiten. La verificación cubre dos casos canónicos: una agregación `AVG` o `SUM` no se aplica sobre una columna de tipo no numérico (descriptor `type_mismatch_aggregate`), y una operación aritmética binaria requiere operandos numéricos en ambos lados (`type_mismatch_arithmetic`). La inferencia opera por categoría abstracta (numérica, textual, fecha, booleana) derivada del tipo declarado en el esquema, con tratamiento permisivo de tipos no resueltos para evitar falsos positivos.

Familia de coherencia cruzada relacional–grafo. En bloques híbridos se verifica que los *labels* de vértice referenciados tengan una tabla relacional asociada según la declaración del grafo de propiedad (descriptor `vertex_label_without_table`). Adicionalmente, en `PathPattern` *multihop* se verifica que el *label* del vértice precedente coincida con el *source label* declarado por la arista, y análogamente para el destino, contemplando las tres direcciones admitidas (`path_step_incoherent`). Esta segunda verificación es una extensión natural de la primera que se incorpora a partir de la observación empírica de un modo de fallo en el cual el modelo construye cadenas *multihop* sintácticamente correctas pero semánticamente incoherentes con la declaración del grafo de propiedad.

Familia de restricciones operativas del motor. Una clase adicional de chequeo, separada conceptualmente del diseño abstracto de la RI, refleja una restricción operativa de DuckPGQ [Wolde et al., 2023]: cada `VertexPattern` debe declarar *label* aunque el estándar lo permita ausente (descriptor `missing_vertex_label`). El verificador detecta esta condición antes de la compilación y emite el descriptor canónico correspondiente, lo que evita que la consulta llegue al motor y falle con un mensaje opaco.

La verificación estática conecta con la tradición de la decodificación restringida inaugurada por PICARD [Scholak and et al., 2022], pero opera a un nivel de abstracción superior en lugar de sobre *tokens*. La política de errores acumula todos los hallazgos antes de devolver, en lugar de cortar al primer fallo: el bucle de retroalimentación recibe así un reporte completo por consulta y puede decidir si re-prompts, repara una porción de la RI o reformula la pregunta.

Verificación dinámica. Si la verificación estática se supera, la consulta se ejecuta sobre una instancia de DuckDB con la extensión DuckPGQ cargada [Wolde et al., 2023]. El motor actúa como *oráculo parcial*: clasifica la consulta como válida, inválida por error de ejecución, o ejecutable pero con salida atípica (cardinalidad cero, presencia inesperada de valores nulos). Los errores y las salidas atípicas generan señales que retroalimentan la construcción de la RI, en línea con la estrategia *execution-guided* documentada en [S. Talaei and et al., 2024, Wenbo Xu, 2025, Yang et al., 2025].

Taxonomía operativa de alucinaciones. La conjunción de los regímenes estático y dinámico define una taxonomía operativa de las alucinaciones que el sistema puede detectar y, en consecuencia, retroalimentar al generador. La taxonomía organiza los descriptores canónicos del verificador en tres niveles, en orden creciente de coste de detección.

Referenciales. Refieren a elementos del esquema dual que no existen: tablas, columnas, etiquetas de vértice o arista, propiedades, calificadores. Los descriptores asociados son `unknown_table`, `unknown_column`, `unknown_qualifier`, `unknown_graph`, `unknown_vertex_label` y `unknown_edge_label`. Su detección es local sobre la RI y no requiere ejecución.

Estructurales. Comprenden las fallas de tipado y de coherencia cruzada entre los componentes relacional y de grafo. Incluyen `type_mismatch_aggregate` y `type_mismatch_arithmetic` en el plano relacional, así como `vertex_label_without_table` y `path_step_incoherent` en el plano híbrido. La detección sigue siendo estática, pero exige consultar el catálogo del grafo de propiedad y, en el caso de las cadenas *multihop*, recorrer la secuencia de pasos.

Semánticas. Cubren los casos en los que la RI es estructuralmente correcta, pero la consulta resultante diverge de la intención del usuario. Su detección es responsabilidad del régimen dinámico: errores de ejecución no anticipados, cardinalidad anómala o discrepancias respecto a relaciones metamórficas conocidas [Chen et al., 1998, Chi, 2025]. El régimen estático no las decide; el dinámico las acota.

Esta taxonomía cumple dos funciones operativas. Provee un vocabulario compartido para reportar fallas en los experimentos del Capítulo 5, y delimita el alcance de cada componente del verificador, lo que evita atribuir al generador fallas que pertenecen al motor o viceversa.

Política de retroalimentación estructurada. El bucle no reenvía al LLM el mensaje de error crudo del motor, como es habitual en esquemas de autocorrección. La señal se traduce a un descriptor categórico que se inyecta como contexto adicional en la etapa de construcción de la RI. Esta mediación es consistente con el principio P4 y con la taxonomía de errores propuesta por [Hong et al., 2026].

4.5. Implementación y replicabilidad

Esta sección documenta la estrategia de implementación, las versiones específicas del stack técnico, la estructura del prototipo, la procedencia de cada componente y el estado actual del trabajo.

4.5.1. Stack técnico y versiones declaradas

El prototipo se ejecuta sobre el siguiente stack, fijado para satisfacer el requisito R8 (reproducibilidad operacional):

Sistema operativo. macOS, arquitectura `arm64`. Se documentan posibles incompatibilidades con otros SO a medida que aparecen.

Gestor de entornos. `uv` (Astral), versión 0.11 o superior, con Python 3.11.15. Esta elección sustituye a `conda`, considerada inicialmente.

Motor de ejecución. DuckDB versión 1.4.4, con la extensión community `duckpgq`, la decisión de la versión se debe a que esta extensión *no* está publicada para DuckDB 1.5.x o superiores sobre `osx_arm64` al momento de redacción.

Modelo de lenguaje inicial. Anthropic Claude Haiku 4.5 (`claude-haiku-4-5`). El modelo se considera intercambiable conforme al principio P5; su elección inicial responde a costo, latencia y disponibilidad de credenciales. Se prevé evaluar Claude Sonnet 4.6, GPT 4, Gemini 3 y modelos experimentales en la fase de experimentos reportables.

Análisis sintáctico de SQL. sqlglot, configurado con dialecto `duckdb`.

Marco de evaluación. NL2SQL360 [B. Li and et al., 2024] para métricas canónicas sobre Spider y BIRD; un módulo propio para métricas del régimen híbrido.

4.6. Resultados esperados

La naturaleza incremental de la propuesta exige separar de forma explícita los resultados que se esperan al cierre del semestre académico en curso de los que constituyen la entrega final de la tesis. Cada conjunto se enuncia en términos verificables —artefactos, evidencia experimental y documentos— y se acompaña de criterios de éxito y limitaciones reconocidas, en consonancia con la transparencia metodológica que ha orientado el resto del capítulo. Los plazos siguen el cronograma de actividades de la tesis (Cuadro 1.1).

4.6.1. Resultados esperados al cierre del semestre 2026-1

El cronograma sitúa el cierre del primer semestre académico en la *Exposición de Avances* prevista para la semana del 28 de junio al 4 de julio de 2026, con un avance acumulado del 80 %. Para esa fecha se espera entregar los siguientes resultados.

Artefactos de software. Se entregará un prototipo funcional capaz de procesar un subconjunto del flujo descrito en la Sección 4.3. En concreto:

1. Un *stack* reproducible documentado (Sección 4.5.1), con las versiones de DuckDB, DuckPGQ, Python y demás dependencias fijadas en archivos de *lock*, instalable mediante un procedimiento de pasos finitos sobre `macOS arm64`.
2. Una versión preliminar de IR-SQL/PGQ que cubra el régimen relacional puro y al menos un subconjunto representativo de los bloques de grafo (`GraphMatch` con patrones lineales y `PropertyProjection`), suficiente para expresar las consultas del corpus piloto descrito a continuación.
3. Un compilador determinista que traduzca este subconjunto de IR-SQL/PGQ a consultas SQL/PGQ ejecutables sobre DuckDB con DuckPGQ, conforme a la restricción operativa del motor descrita en la Sección 4.4.3.
4. Un verificador estático que amplíe el alcance referencial inicial al esquema dual, cubriendo al menos: existencia de tablas, columnas, etiquetas y propiedades; consistencia de aliases; y verificación de que toda variable de `GraphMatch` esté declarada antes de su uso.
5. Una primera versión del bucle de retroalimentación con descriptores categóricos, integrada con la verificación dinámica sobre DuckPGQ.

Corpus piloto y evidencia experimental. Se entregará un corpus piloto de evaluación, construido específicamente para el régimen SQL/PGQ y de tamaño suficiente para validar la mecánica del *pipeline*, sin pretender aún la cobertura empírica que la entrega final exigirá. La evidencia experimental asociada incluirá:

1. Un experimento ampliado sobre Spider, con un volumen de preguntas suficiente para que la tasa de detección del verificador estático refleje el comportamiento del modelo y no la ausencia de alucinaciones en muestras pequeñas.
2. Un experimento inicial sobre el corpus piloto SQL/PGQ, que reporte tasas de validez sintáctica, validez referencial y ejecutabilidad sobre DuckPGQ. Las métricas se reportarán por separado para los regímenes relacional puro, de grafo puro e híbrido.
3. Un análisis de errores por categoría (referenciales, estructurales, semánticas), siguiendo la tipología introducida en la Sección 4.4.4.

Documento. El documento de tesis al cierre del semestre incluirá los Capítulos 1 a 4 consolidados, una primera versión del Capítulo 5 con el diseño experimental, y resultados preliminares del experimento sobre Spider y del corpus piloto. La presentación de Avances acompañará al documento como soporte para la exposición ante el jurado.

4.6.2. Resultados esperados al cierre de la tesis

El cronograma sitúa la sustentación final en la semana del 1 al 12 de diciembre de 2026, con un avance acumulado del 100 %. Para esa fecha se espera consolidar los siguientes resultados.

Artefactos de software completos. La entrega final extenderá el prototipo del semestre 2026-1 con:

1. La especificación completa de IR-SQL/PGQ —incluida la familia `HybridComposition` y el conjunto exhaustivo de reglas de tipado— y el compilador determinista correspondiente, validado sobre todas las construcciones del corpus de evaluación.

2. El bucle de verificación integrado en sus dos regímenes (estático y dinámico) con la política de retroalimentación estructurada operativa, capaz de iterar hasta el criterio de terminación definido en la Sección 4.3.

3. La interfaz abstracta de LLM que materializa el principio P5, instanciada al menos sobre dos modelos distintos (Anthropic Claude Haiku 4.5 y un segundo modelo aún por fijar entre Claude Sonnet 4.6 y un modelo de la familia `XiYanSQL-QwenCoder`) para evidenciar la sustitución sin modificación arquitectónica.

Corpus SQL/PGQ de evaluación. Se entregará un corpus de evaluación más amplio que el corpus piloto, construido con el procedimiento descrito en el Capítulo 5. El corpus incluirá consultas en los tres regímenes —relacional puro, de grafo puro e híbrido—, con su esquema dual asociado y la consulta de referencia (*gold*). La construcción se documentará de modo que un tercero pueda regenerarlo a partir de las mismas fuentes.

Evidencia experimental consolidada. La validación final reportará:

1. **Métricas canónicas** sobre SPIDER y BIRD mediante NL2SQL360 [B. Li and et al., 2024]: *Execution Accuracy*, *Exact Match* y *Valid Efficiency Score*, en condiciones comparables con los *baselines* declarados en el Capítulo 5.

2. **Métricas propias del régimen híbrido** —no cubiertas por NL2SQL360—, que evalúen la consistencia cruzada entre componentes relacional y de grafo, la cobertura de los patrones `GRAPH_TABLE` y la robustez ante perturbaciones controladas del esquema dual.

3. **Estudios de ablación** que aíslen la contribución de cada componente de la propuesta: el efecto del anclaje al esquema dual, el de la mediación por RI respecto a la generación directa, y el de la verificación en cascada respecto a la verificación post-ejecución únicamente.

4. **Análisis comparativo contra *baselines*** representativos de las cuatro líneas identificadas en el Capítulo 3, restringido a los *baselines* cuyo código y datos sean accesibles bajo las condiciones de reproducibilidad declaradas.

5. **Análisis de errores categorizado** sobre el espacio completo de la taxonomía de alucinaciones introducida en la Sección 4.4.4, con discusión explícita de las limitaciones de cada categoría de verificación.

Resultado científico. Más allá de los artefactos y de la evidencia empírica, la tesis aspira a sostener tres aportes científicos verificables:

1. Una *caracterización formal de la RI IR-SQL/PGQ* —gramática, sistema de tipos, función de compilación— que sirva como referencia para trabajos posteriores que extiendan la generación a otros dialectos del estándar de grafos.
2. *Evidencia empírica* de que la mediación por RI tipada con verificación en cascada reduce las alucinaciones referenciales y estructurales en el régimen híbrido respecto a la generación directa, en condiciones controladas.
3. Un *procedimiento de construcción de corpus* SQL/PGQ que cubra los tres regímenes y que pueda reutilizarse en investigaciones posteriores.

Documentos y divulgación. La entrega final incluirá:

1. El documento de tesis completo con los seis capítulos cerrados, conclusiones, limitaciones reconocidas y trabajo futuro.
2. Un reporte técnico o un borrador de artículo derivado de los resultados, según la oportunidad de divulgación que se identifique en la fase de síntesis.
3. El repositorio público con el prototipo, las trazas de los experimentos reportados y los archivos de configuración necesarios para regenerarlos.

4.6.3. Criterios de éxito y limitaciones reconocidas

La propuesta se considerará exitosa al cierre de la tesis si se satisfacen los siguientes criterios, formulados de manera que admitan verificación independiente.

Éxito mínimo. El prototipo procesa, sin intervención humana, consultas en los tres regímenes del corpus de evaluación, produce SQL/PGQ ejecutable sobre DuckPGQ, y la verificación estática rechaza correctamente al menos las alucinaciones referenciales sobre un conjunto controlado de preguntas con señuelos de esquema.

Éxito esperado. Los criterios anteriores se cumplen y, además, la mediación por RI mejora —en métricas canónicas o propias— al menos un *baseline* representativo de cada una de las líneas del Capítulo 3, en condiciones de reproducibilidad.

Éxito amplio. Se cumplen los criterios anteriores y los resultados se traducen en una contribución divulgable en formato de artículo o reporte técnico revisable.

Tres limitaciones se reconocen explícitamente, sin que su presencia invalide los resultados esperados:

1. La cobertura de la RI no abarcará la totalidad de SQL/PGQ; las construcciones fuera del subconjunto soportado se documentarán como trabajo futuro.

2. La verificación dinámica está acotada por las capacidades de DuckPGQ en la versión declarada en la Sección 4.5.1; las construcciones del estándar que la extensión no soporte se reportarán como restricciones operativas.

3. Los *baselines* comparativos se restringirán a sistemas con código accesible bajo licencias compatibles; comparaciones con sistemas comerciales sin acceso al código se discutirán cualitativamente, no se reportarán como evidencia empírica directa.

4.7. Síntesis y transición

Este capítulo describió una arquitectura que articula cuatro componentes —anclaje al esquema dual, RI tipada IR-SQL/PGQ, compilador determinista y bucle de verificación— bajo un *pipeline* de cinco etapas y cinco principios de diseño. La propuesta responde de forma explícita a los ocho requisitos derivados del planteamiento del problema y se apoya en una base experimental modular que combina elementos reutilizados del estado del arte (XiYan-SQL, CHESS, RSL-SQL, DuckDB+DuckPGQ, NL2SQL360, sqlglot) con código propio cuya contribución central es la RI IR-SQL/PGQ, su compilador determinista y el verificador estructurado.

La presentación se hizo bajo dos exigencias metodológicas: la *trazabilidad de la procedencia* —toda pieza reutilizada se identifica con su origen y la modificación que se le aplica se declara explícitamente— y la *transparencia experimental* —los pivotes y aprendizajes operacionales se documentan en lugar de ocultarse. Esta orientación responde al carácter incremental del trabajo y a la recomendación de que la propuesta pueda ser replicada de forma independiente. Los resultados esperados se han enunciado en términos verificables, con plazos derivados del cronograma y con criterios de éxito que admiten validación externa.

El siguiente capítulo describe el diseño experimental que permite validar la propuesta: los *benchmarks* empleados, la construcción del corpus SQL/PGQ de evaluación, las métricas que complementan a las canónicas y los *baselines* comparativos sobre los que se contrasta el desempeño de la arquitectura.

Capítulo 5

Experimentos

Capítulo 6

Resultados

Capítulo 7

Conclusiones y Trabajos Futuros

Las conclusiones de la tesis son una parte muy importante y tiene las siguientes partes.

Escribir las conclusiones de su trabajo por cada uno de los objetivos específicos definidos en el Capítulo 1 y luego escribir la conclusión general de tu trabajo.

7.1. Problemas encontrados

La segunda parte de este capítulo corresponde a los problemas encontrados, esta sección es muy importante para que los siguientes estudiantes o investigadores que hagan algo en esta línea no cometan los mismos errores y tu tesis sea un buen peldaño para avanzar más rápido.

7.2. Recomendaciones

En esta sección el tesista debe reflejar que la tesis ha permitido adquirir nuevos conocimientos que podrían servir para guiar otros trabajos en el futuro.

7.3. Trabajos futuros

En base a los puntos anteriores es recomendable que tu tesis también sugiera trabajos futuros. Esta sección es esecialmente útil para otras ideas de tesis.

Todo este capítulo no debe ser más de unas 4 páginas.

Bibliografía

[A. Bazaga and et al., 2023] A. Bazaga, P. L. and et al. (2023). Sqlformer: Deep autoregressive query graph generation for text-to-sql translation.

[Adedeji and Mabere, 2023] Adedeji and Mabere, M. (2023). From data to decisions: Llms redefining business intelligence author. *Journal of Artificial Intelligence*, 6:14.

[Angles et al., 2017] Angles, R., Arenas, M., Barceló, P., Hogan, A., Reutter, J., and Vrgoč, D. (2017). Foundations of modern query languages for graph databases. *ACM Computing Surveys*, 50(5).

[AtScale, Inc., 2024] AtScale, Inc. (2024). *Semantic Modeling Language (SML) Specification v0.1*. AtScale. Technical Specification for Open Source Semantic Modeling.

[B. Li and et al., 2024] B. Li, Y. Luo, C. C. and et al. (2024). The dawn of natural language to sql: Are we fully ready?

[B. Li and Luo, 2025] B. Li, J. Zhang, J. F. Y. X. C. C. N. T. and Luo, Y. (2025). Alpha-sql: Zero-shot text-to-sql using monte carlo tree search. *ICML 2025*.

[B. Wang and et al., 2023] B. Wang, C. Ren, J. Y. and et al. (2023). Mac-sql: A multi-agent collaborative framework for text-to-sql. *COLING 2025*.

[B. Wang and et al., 2021] B. Wang, R. Shin, X. L. and et al. (2021). Rat-sql: Relation-aware schema encoding and linking for text-to-sql parsers. *ACL 2020*.

[Bonifati et al., 2025] Bonifati, A. et al. (2025). On the expressiveness of languages for querying property graphs in relational databases. *arXiv:2510.07062*.

[Brown et al., 2020] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[C. Gao and et al., 2022] C. Gao, B. Li, W. Z. and et al. (2022). Towards generalizable and robust text-to-sql parsing. *EMNLP 2022 Findings*.

[Chang and Fosler-Lussier, 2023] Chang, S. and Fosler-Lussier, E. (2023). Selective demonstrations for cross-domain text-to-sql. *Findings of EMNLP 2023*.

[Chen et al., 1998] Chen, T. Y., Cheung, S. C., and Yiu, S. M. (1998). Metamorphic testing: A new approach for generating next test cases. Technical Report HKUST-CS98-01, Department of Computer Science, Hong Kong University of Science and Technology.

[Chi, 2025] Chi, H. (2025). Reducing hallucinations in text-to-sql building trust and accuracy in data access.

[Codd, 1970] Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387.

[D. Gao and et al., 2023] D. Gao, H. Wang, Y. L. and et al. (2023). Text-to-sql empowered by large language models: A benchmark evaluation. *PVLDB 2024*.

[Deutsch et al., 2022] Deutsch, A., Francis, N., Green, A., Hare, K., Li, B., Libkin, L., Lindaaker, T., Marsault, V., Martens, W., Michels, J., Murlak, F., Plantikow, S., Selmer, P., van Rest, O., Voigt, H., Vrgoč, D., Wu, M., and Zemke, F. (2022). Graph pattern matching in GQL and SQL/PGQ. In *Proc. ACM SIGMOD International Conference on Management of Data*, pages 2246–2258.

[Ding et al., 2026] Ding, Z., Lin, Y., Zeng, T., Zhu, R., Ding, B., and Zhou, J. (2026). Ambisql: Interactive ambiguity detection and resolution for text-to-sql.

[Dong et al., 2024] Dong, Y., Ding, J., Jiang, X., Li, G., Li, Z., and Jin, Z. (2024). Codescore: Evaluating code generation by learning code execution.

[et al., 2023] et al., M. K. (2023). Crush4sql: Collective retrieval using schema hallucination for text2sql. *EMNLP 2023*.

[et al., 2025] et al., S. G. (2025). Blendsql: A scalable dialect for unifying hybrid question answering in relational algebra. *ACL 2024 Findings*.

[et al., 2024] et al., Z. C. (2024). Rsl-sql: Robust schema linking in text-to-sql generation.

[Francis et al., 2023] Francis, N., Gheerbrant, A., Guagliardo, P., Libkin, L., Marsault, V., Martens, W., Murlak, F., Peterfreund, L., Rogova, A., and Vrgoč, D. (2023). GPC: A pattern calculus for property graphs. In *Proc. 42nd ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS)*, pages 241–250.

[Fürst et al., 2024] Fürst, J., Kosten, C., Nooralahzadeh, F., Zhang, Y., and Stockinger, K. (2024). Evaluating the data model robustness of text-to-sql systems based on real user queries.

[Gan et al., 2021] Gan, Y., Chen, X., Xie, J., Purver, M., Woodward, J. R., Drake, J., and Zhang, Q. (2021). Natural SQL: Making SQL easier to infer from natural language specifications. In *Findings of EMNLP 2021*, pages 2030–2042.

[Gan and et al., 2023] Gan, Y. and et al. (2023). Improving generalization in language model-based text-to-sql semantic parsing: Two simple semantic boundary-based techniques. *ACL 2023*.

[Guo et al., 2019] Guo, J., Zhan, Z., Gao, Y., Xiao, Y., Lou, J.-G., Liu, T., and Zhang, D. (2019). Towards complex text-to-SQL in cross-domain database with intermediate representation. In *Proc. 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4524–4535.

[H. Fu and et al., 2023] H. Fu, C. Liu, B. W. and et al. (2023). Catsql : Towards real world natural language to sql applications. *PVLDB 2023*.

[H. Li and Chen, 2023] H. Li, J. Zhang, C. L. and Chen, H. (2023). Resdsql: Decoupling schema linking and skeleton parsing for text-to-sql. *AAAI 2023*.

[H. Li and et al., 2024] H. Li, J. Zhang, H. L. and et al. (2024). Codes: Towards building open-source language models for text-to-sql. *SIGMOD 2024*.

[H. Zhang and et al., 2024] H. Zhang, R. Cao, H. X. and et al. (2024). Coe-sql: In-context learning for multi-turn text-to-sql with chain-of-editions. *NAACL 2024*.

[Herzig et al., 2021] Herzig, J., Shaw, P., Chang, M.-W., Guu, K., Pasupat, P., and Zhang, Y. (2021). Unlocking compositional generalization in pre-trained models using intermediate representations. In *arXiv:2104.07478*.

[Hong et al., 2026] Hong, Z., Chen, H., Yuan, Z., Zhang, Q., Zhuang, L., Liao, Q., Huang, F., Song, Y., and Huang, X. (2026). Errorllm: Modeling sql errors for text-to-sql refinement.

[Hong et al., 2024] Hong, Z., Yuan, Z., Zhang, Q., Chen, H., Dong, J., Huang, F., and Huang, X. (2024). Next-generation database interfaces: A survey of LLM-based text-to-SQL. *arXiv:2406.08426*.

[Hui and et al., 2022] Hui, B. and et al. (2022). S2sql: Injecting syntax to question-schema interaction graph encoder for text-to-sql parsers. *ACL 2022 Findings*.

[ISO/IEC JTC 1/SC 32, 2023] ISO/IEC JTC 1/SC 32 (2023). Information technology – database languages – sql – part 16: Property graph queries (sql/pgq). Standard, International Organization for Standardization, Geneva, CH.

[J. Huang and et al., 2021] J. Huang, Y. Wang, Y. W. and et al. (2021). Relation aware semi-autoregressive semantic parsing for nl2sql.

[J. Qi and et al., 2022] J. Qi, J. Tang, Z. H. and et al. (2022). Rasat: Integrating relational structures into pretrained seq2seq model for text-to-sql. *EMNLP 2022*.

[J. Shi and et al., 2025] J. Shi, Bo Xu, J. L. and et al. (2025). Gen-sql: Efficient text-to-sql by bridging natural language question and database schema with pseudo-schema. *COLING 2025*.

[Ji et al., 2023] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y., Madotto, A., and Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12).

[L. Dou and et al., 2022] L. Dou, Y. Gao, X. L. and et al. (2022). Importance of synthesizing high- quality data for text-to-sql parsing.

[L. Zeng and Hakkani-Tur, 2022] L. Zeng, S. H. K. P. and Hakkani-Tur, D. (2022). N-best hypothe- ses reranking for text-to-sql systems.

[Lei et al., 2025] Lei, F., Su, J., Chen, Y., Li, Q., Zhang, Z.-X., et al. (2025). Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*. Oral Presentation.

[Li and et al., 2022] Li, J. and et al. (2022). Graphix-t5: Mixing pre-trained transformers with graph-aware layers for text-to-sql parsing.

[Li et al., 2023] Li, J., Hui, B., Qu, G., Yang, J., Li, B., Li, B., Wang, B., Qin, B., Geng, R., Huo, N., Zhou, X., Ma, C., Li, G., Chang, K. C.-C., Huang, F., Cheng, R., and Li, Y. (2023). Can LLM already serve as a database interface? a BIg bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[Li and et al., 2024] Li, Z. and et al. (2024). Pet-sql: A prompt-enhanced two-round refinement of text-to-sql with cross-consistency.

[Lin and et al., 2020] Lin, X. V. and et al. (2020). Bridging textual and tabular data for cross-domain text-to-sql semantic parsing.

[Liu et al., 2025a] Liu, M., Wang, B., Chen, W., Lan, T., Yan, C., Liu, H., Hu, Y., Yan, X., and Cui, B. (2025a). NLI4DB: A systematic review of natural language interfaces for databases. *arXiv:2503.02435*.

[Liu et al., 2025b] Liu, R., Chen, X., Zhang, J., Zhang, Q., Zhang, Y., and Yang, B. (2025b). Safenlidb: A privacy-preserving safety alignment framework for llm-based natural language database interfaces.

[Liu et al., 2024] Liu, X., Shen, S., Li, B., Ma, P., Jiang, R., Zhang, Y., Fan, J., Li, G., Tang, N., and Luo, Y. (2024). A survey of NL2SQL with large language models: Where are we, and where are we going? *arXiv:2408.05109*.

[Lyu et al., 2026] Lyu, S., Ban, L., Wu, Z., Luo, T., Liu, J., Ma, C., Luo, Y., Tang, N., Qi, S., Lin, H., Liu, Y., and Hong, C. (2026). Text2gql-bench: A text to graph query language benchmark [experiment, analysis & benchmark].

[M. Pourreza and et al., 2025] M. Pourreza, H. Li, R. S. and et al. (2025). Chase-sql: Multi-path reasoning and preference optimized candidate selection in text-to-sql. *ICLR 2025*.

[Nie et al., 2022] Nie, L., Cao, S., Shi, J., Sun, J., Tian, Q., Hou, L., Li, J., and Zhai, J. (2022). GraphQ IR: Unifying the semantic parsing of graph query languages with one intermediate representation. In *Proc. EMNLP 2022*, pages 5848–5865.

[Pérez et al., 2025] Pérez, A. S., Boukhary, A., Papotti, P., Lozano, L. C., and Elwood, A. (2025). An LLM-based approach for insight generation in data analysis. In Chiruzzo, L., Ritter, A., and Wang, L., editors, *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 562–582, Albuquerque, New Mexico. Association for Computational Linguistics.

[Pourreza and Rafiei, 2023] Pourreza, M. and Rafiei, D. (2023). Din-sql: Decomposed in-context learning of text-to-sql with self-correction. *NeurIPS 2023*.

[Pourreza and Rafiei, 2024] Pourreza, M. and Rafiei, D. (2024). Dts-sql: Decomposed text-to-sql with small large language models. *EMNLP 2024 Findings*.

[Qin et al., 2022] Qin, B., Hui, B., Wang, L., Yang, M., Li, J., Li, B., Geng, R., Cao, R., Sun, J., Si, L., Huang, F., and Li, Y. (2022). A survey on text-to-SQL parsing: Concepts, methods, and future directions. *arXiv:2208.13629*.

[Qinggang Zhang, 2025] Qinggang Zhang, Hao Chen, J. D. S. C. F. H. X. H. (2025). Structure guided large language model for sql generation. *ICML 2024*.

[Qu et al., 2024] Qu, G., Li, J., Li, B., et al. (2024). Before generation, align it! a novel and effective strategy for mitigating hallucinations in text-to-sql generation. *ACL 2024 Findings*.

[Quamar et al., 2022] Quamar, A., Efthymiou, V., Lei, C., and Lei, C. (2022). Natural language interfaces to data. *Foundations and Trends in Databases*, 11(4):319–414.

[Rubin and Berant, 2021] Rubin, O. and Berant, J. (2021). Smbop: Semi-autoregressive bottom-up semantic parsing. *NAACL 2021*.

[S. Talaei and et al., 2024] S. Talaei, M. Pourreza, Y.-C. C. and et al. (2024). Chess: Contextual harnessing for efficient sql synthesis. *ICML*.

[Safdarian et al., 2025] Safdarian, A., Mohammadi, M., Jahanbakhsh, E., Naderi, M. S., and Faili, H. (2025). Schemagraphsql: Efficient schema linking with pathfinding graph algorithms for text-to-sql on large-scale databases.

[Scholak and et al., 2022] Scholak, T. and et al. (2022). Picard: Parsing incrementally for constrained auto-regressive decoding from language models. *EMNLP 2021*.

[Shaoming Duan, 2025] Shaoming Duan, Zirui Wang, C. L. Z. Z. Y. Z. P. H. L. Y. Z. P. (2025). Cred-sql: Enhancing large scale database text-to-sql.

[Shi et al., 2025] Shi, L., Tang, Z., Zhang, N., Zhang, X., and Yang, Z. (2025). A survey on employing large language models for text-to-SQL tasks. *ACM Computing Surveys*.

[Stoisser et al., 2025] Stoisser, J. L., Martell, M. B., Phillips, L., Hansen, C., and Fauqueur, J. (2025). Struct-llm: Unifying tabular and graph reasoning with reinforcement learning for semantic parsing. *ICLR 2025*.

[T. Ren and et al., 2024] T. Ren, Y. Fan, Z. H. and et al. (2024). Purple: Making a large language model a better sql writer. *ICDE 2024*.

[Tang et al., 2026] Tang, Z., Zhang, Y., Cai, S., and Wang, R. (2026). Llm-fk: Multi-agent llm reasoning for foreign key detection in large-scale complex databases.

[Vaswani et al., 2017] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[Victor Zhong, 2017] Victor Zhong, Caiming Xiong, R. S. (2017). Seq2sql: Generating structured queries from natural language using reinforcement learning.

[Wenbo Xu, 2025] Wenbo Xu, Haifeng Zhu, L. Y. C. L. P. H. S. D. J. Z. P. (2025). Ts-sql: Test-driven self-refinement for text-to-sql. *EMNLP 2025 Findings*.

[Wolde et al., 2023] Wolde, D. t., Szárnyas, G., and Boncz, P. (2023). Duckpgq: Bringing sql/pgq to duckdb. *Proc. VLDB Endow.*, 16(12):4034–4037.

[X. Dong and et al., 2023] X. Dong, C. Zhang, Y. G. and et al. (2023). C3: Zero-shot text-to-sql with chatgpt.

[X. Xie and Guo, 2025] X. Xie, G. Xu, L. Z. and Guo, R. (2025). Opensearch-sql: Enhancing text-to-sql with dynamic few-shot and consistency alignment. *ACM on Management of Data.*

[Xie and et al., 2024] Xie, Y. and et al. (2024). Decomposition for enhancing attention: Improving llm-based text-to-sql through workflow paradigm. *ACL 2024 Findings.*

[Xutao Mao, 2025] Xutao Mao, Tao Liu, H. Z. (2025). Steinersql: Graph-guided mathematical reasoning for text-to-sql generation.

[Y. Fan and et al., 2024] Y. Fan, Z. He, T. R. and et al. (2024). Metasql: A generate-then-rank framework for natural language to sql translation.

[Y. Qin and et al., 2025] Y. Qin, C. Chen, Z. F. and et al. (2025). Route: Robust multitask tuning and collaboration for text-to-sql. *ICLR 2025.*

[Y. Xiang and et al., 2023] Y. Xiang, Q.-W. Zhang, X. Z. and et al. (2023). G3r: A graph-guided generate-and-rerank framework for complex and cross-domain text-to-sql generation. *ACL 2023 Findings.*

[Yang et al., 2025] Yang, B., Xia, Y., Sun, W., and Liu, Y. (2025). Hallucination detection for llm-based text-to-sql generation via two-stage metamorphic testing.

[Yaxun Dai, 2025] Yaxun Dai, Haiqin Yang, M. H. P. C. (2025). Parsql: Enhancing text-to-sql through parsing and reasoning. *ACL 2025 Findings.*

[Yingqi Gao, 2024] Yingqi Gao, Yifu Liu, X. L. X. S. Y. Z. Y. W. S. L. W. L. Y. H. Z. L. J. G. L. M. Y. L. (2024). Xiyan-sql: A multi-generator ensemble framework for text-to-sql.

[Z. Gu and et al., 2024] Z. Gu, J. F. and et al. (2024). Interleaving pre-trained language models and large language models for zero-shot nl2sql generation.

[Z. Gu and et al., 2023] Z. Gu, J. Fan, N. T. and et al. (2023). Select-sql: Self-correcting ensemble chain-of-thought for text-to-sql.

[Zelle and Mooney, 1996] Zelle, J. M. and Mooney, R. J. (1996). Learning to parse database queries using inductive logic programming. In *Proceedings of the Thirteenth National Conference on Artificial Intelligence (AAAI)*, pages 1050–1055.

[Zhang and et al., 2024] Zhang, C. and et al. (2024). Finsql: Model-agnostic llms-based text-to-sql framework for financial analysis.

[Zhang and et al., 2023] Zhang, H. and et al. (2023). Act-sql: In-context learning for text-to-sql with automatically-generated chain-of-thought. *Findings of EMNLP 2023.*